MASTERARBEIT · FORSCHUNGS- UND IMPLEMENTIERUNGSSTAND

Hybride photonische Sortierung

Gesamtdokumentation der Masterarbeit

Recherche, mathematische Modelle, Implementierung und Evaluierung

Bharadwaj Srikumar

Master Informatik · Hochschule Bochum

Betreuung: Prof. Dr. Henrik Blunck

Stand: 24. Oktober 2026

Arbeitstitel

Mapping Hybrid Optoelectronic Sorting Architectures onto Modern Photonic Accelerators: A Hardware-Aware Evaluation and Simulation

Diese Dokumentation beschreibt den erreichten Stand der Masterarbeit. Sie verbindet Literatur, mathematische Herleitung, ausführbaren Code und gespeicherte Ergebnisse. Reale Hardwarevalidierung, photonische Laufzeitvorteile und Energieeffizienz sind noch offen.

Inhaltsverzeichnis

Zusammenfassung 3

1. Forschungsfrage und Abgrenzung 4

2. Grundlagen: von Zahlen zu optischen Signalen 5

3. Literaturauswertung und Konsequenzen für das Mapping 8

4. Komplexitätsmodell C = (T, S, H, D) 11

5. Die fünf untersuchten Sortierarchitekturen 14

6. Mathematische Mappings und stabile Datensätze 17

7. Q.ANT-SDK und periodischer Vergleich 21

8. Referenzversuch: Eingaben, Quantisierung und Rauschen 25

9. Ein gemeinsames Signal- und Rauschmodell 27

10. Qualitätsmetriken: vollständige Ordnung und brauchbare Rangfolge 30

11. Ergebnisse aus den gespeicherten Versuchen 32

12. Aufrufe, Datenbewegung und mögliche Systemkosten 37

13. Vom Konzept zum ausführbaren Code 39

14. Reproduzierbarkeit und Prüfung 41

15. Hardwarefragen und noch ausstehende Arbeit 43

16. Gesamtbewertung des aktuellen Stands 45

Anhang A. Literaturverzeichnis und Einordnung 46

Anhang B. Quellenbestand und Ergebnisprovenienz 52

Anhang C. Begriffe und Leseschlüssel 55

Zusammenfassung

Die Arbeit untersucht, unter welchen Genauigkeits- und Ressourcenbedingungen vollständige, stabile Sortierung sinnvoll auf hybride photonische Prozessoren abgebildet werden kann. Entscheidend ist der gesamte Ablauf: photonische Verarbeitung, diskrete Entscheidungen, Gleichstände, Datenbewegung und elektronische Ausgabe.

Aus fünf untersuchten Architekturen werden Bitonic und Rangsortierung vertieft. Bitonic verarbeitet relativ wenige Vergleiche in abhängigen Stufen; Rangsortierung bündelt eine quadratische Paarphase. Das unveränderte Modell C = (T, S, H, D) trennt skalare sequenzielle Gesamtarbeit, logischen Speicher, installierte Verarbeitungspositionen und unregenerierte optische Tiefe. T ist eine real-RAM-artige Operationszahl, keine physische Sekundenzahl.

Gespeicherte Eingaben, erschöpfende 4-/8-Bit-Paarprüfungen und vollständige Sortierläufe stützen die mathematische und CPU-seitige Funktionsfähigkeit. Der periodische SDK-Baustein mit Phasenbildung auf dem Host entscheidet nach Referenzkorrektur korrekt. Im vorhandenen Ablauf liegt seine Ausgangsdifferenz jedoch bereits auf dem Host; der zusätzliche Aufruf bietet dort keinen nachgewiesenen Entscheidungsnutzen.

Eine separate affine Phasenbildung innerhalb der MVM ist mathematisch äquivalent, verliert im geprüften CPU-Backend jedoch Präzision: 40 von 65.536 geordneten 8-Bit-Paaren werden ohne Zusatzrauschen fälschlich gleich. Die öffentliche Schnittstelle liefert auch dabei den Zwischenwert an den Host zurück. Dieser Befund begrenzt die konkrete Variante; er belegt keine generelle Hardwaregrenze.

Zusätzliche Qualitätsmetriken zeigen den Unterschied zwischen vollständiger Ordnung und brauchbarer Rangfolge: Bei 8 Bit, N = 256, verschiedenen Schlüsseln und η = 0,25 sind im Referenzmodell nur 5,9 % der Bitonic-Ausgaben vollständig korrekt, aber das mittlere Kendall-τ-b beträgt 0,999826 und Recall@10 99,88 %. Die Fehler betreffen überwiegend benachbarte Schlüssel.

Das gemeinsame Rauschmodell macht Signalabstand, Quantisierung und Störort vergleichbar. Seine Parameter sind Sensitivitätsannahmen, keine Q.ANT-Kalibrierung. Eine digitale CPU-Baseline ist vorhanden; GPU-, NPU- und Energiemessungen stehen aus. Der Bericht dokumentiert diese Grenzen ebenso wie die Ergebnisse und die fünf fehlenden historischen Ausgabekonfigurationen.

Tabelle 1. Erreichter Stand und Aussagegrenzen

Bereich	Vorhandene Evidenz	Offen
Literatur und Modell	Fünf Architekturen; C = (T,S,H,D); hergeleitete Schranken	Keine universelle neue Komplexitätstheorie
Implementierung	Bitonic, Rangsortierung, lineares und periodisches CPU-SDK	Nützliche hostfreie Kaskade
Qualität und Rauschen	Referenz, τ, Abstandsfehler, Recall; gemeinsame Kontrollen	Gerätespezifische Kennlinie, Rauschen und Drift
Leistung	Stabile digitale CPU-Baseline	GPU, vollständige NPU-Zeit und Energie

1. Forschungsfrage und Abgrenzung

1.1 Warum Sortieren auf photonischen Prozessoren?

Sortierung ist eine grundlegende Operation der Datenverarbeitung. Die Aufgabe verbindet eine einfach beschreibbare Ordnung mit vielen Vergleichen und Datenbewegungen. Gerade dadurch eignet sie sich zur Prüfung, ob ein photonischer Rechenkern außerhalb dichter neuronaler Matrixoperationen einen nützlichen Beitrag leisten kann. Ein mathematisch ausdrückbarer Kernel ist noch kein effizientes Systemmapping: Entscheidend ist, welche Arbeit er übernimmt und welchen Aufwand seine Einbettung verursacht.

Bitonic besitzt einen festen, datenunabhängigen Vergleichsplan. Die Rangsortierung kann ihre Paarvergleiche unabhängig vorbereiten. Beide Strukturen erscheinen daher zunächst parallelisierbar. Sie stellen jedoch gegensätzliche Anforderungen: Bitonic benötigt relativ wenige Vergleiche in mehreren abhängigen Stufen; Rangsortierung benötigt wesentlich mehr Vergleiche, kann diese aber breit bündeln. Diese Gegenüberstellung bildet den Kern der Arbeit. [6-10]

Die zentrale Frage lautet: **Unter welchen Genauigkeits-, Datenbewegungs- und Ressourcenbedingungen lassen sich Bitonic- und Rangsortierung sinnvoll auf hybride photonische Prozessoren abbilden?** Ein mögliches negatives Ergebnis, etwa dass Schnittstellen oder elektronische Verarbeitung den Nutzen aufheben, ist eine fachlich relevante Antwort.

1.2 Konkrete Teilfragen

- Welche Teile eines stabilen Vergleichs und der vollständigen Sortierung sind lineare Transformation, nichtlineare Entscheidung, Akkumulation oder Datenbewegung?
- Welche Aussagen liefern die fünf historischen beziehungsweise hybriden Architekturen unter einer einheitlichen Definition von T, S, H und D?
- Wie beeinflussen Signalabstand, Quantisierung, Rauschen und Gleichstände die vollständige Sortierkorrektheit und die Qualität einer näherungsweisen Rangfolge?
- Welche Aufrufe, logischen Datenmengen und Hostoperationen verursacht ein SDK-Mapping gegenüber einer direkten elektronischen Entscheidung?
- Welche Hardwareeigenschaften müssten gegeben sein, damit ein vollständiger hybrider Ablauf die gemessene digitale Baseline erreichen oder übertreffen könnte?

1.3 Was die Arbeit untersucht

Der Literaturteil umfasst Analog Photonic Computing, programmierbare photonische Schaltungen, hybride Präzision, Konversion, Rauschen, Regeneration, Energie und fünf Sortierarchitekturen. Die ausführbare Untersuchung konzentriert sich auf ganzzahlige 4-/8-Bit-Schlüssel, Bitonic und Rangsortierung. Die vorliegenden Versuche prüfen mathematische Kontrollen, CPU-Softwareintegration, synthetische Signalmodelle und eine digitale CPU-Baseline.

Q.ANT dient als konkrete Referenz für einen öffentlich einsehbaren SDK-Pfad mit periodischer Nichtlinearität. Eine Envise-artige Trennung aus photonischer Matrixoperation und digitaler Weiterverarbeitung dient als Architekturkontrast. Die historischen Sorter werden funktional ausgewertet; ihre optischen Bauelemente, räumlichen Layouts und Taktnetze werden nicht als digitale Zwillinge nachgebaut.

Eine Energieüberlegenheit und unbegrenzte Kaskadierbarkeit werden nicht vorausgesetzt. Chipentwicklung und universelle Komplexitätsbeweise sind nicht das Ziel. PRISM ist eine verwandte Auswahlaufgabe, keine vollständige stabile Sortierung. [23]

2. Grundlagen: von Zahlen zu optischen Signalen

2.1 Analog, digital und photonisch sind verschiedene Eigenschaften

Bei digitaler Darstellung werden Werte über diskrete Symbole codiert. Analoge Verarbeitung bildet Werte auf kontinuierliche physikalische Größen ab, etwa Spannung, Feldamplitude, Phase oder Leistung. Photonisch bezeichnet das verwendete physikalische Medium. Ein photonisches System kann digitale optische Bitfolgen übertragen, analoge gewichtete Summen berechnen oder beide Formen mit Elektronik verbinden.

Die Begriffe sollten deshalb nicht gleichgesetzt werden. Die untersuchten historischen Sortierer verwenden teilweise bitserielle optische Daten und elektronische Smart Pixels. Das moderne Vergleichsmodell codiert ganze Schlüssel als normierte Zahlen. Ein FPGA-Radixsortierer mit optischen Links sortiert elektronisch, obwohl seine Kommunikation photonisch ist. [7-11]

Ein real-RAM-Modell ist wiederum eine mathematische Idealisierung mit exakten skalaren Operationen. Es behauptet keine unendliche Auflösung realer Hardware. Die Komplexitätsanalyse und die endlichen numerischen Experimente beantworten unterschiedliche Fragen und werden getrennt geführt.

2.2 Was Licht für Rechenoperationen bereitstellt

Optische Systeme können viele räumliche oder spektrale Kanäle gleichzeitig führen. Interferenz, gewichtete Transmission und Detektion ermöglichen lineare Transformationen. Wellenlängenmultiplexing nutzt mehrere Träger; räumliches Broadcasting verteilt ein Signal auf mehrere Orte. Diese Eigenschaften können breite Operationen unterstützen, erfordern aber passende Quellen, Modulatoren, Empfänger und Datenversorgung. [1, 2, 28]

Ein generischer linearer Kern wird mathematisch als $y = Wx$ beschrieben. Bei einem kohärenten Aufbau können komplexe Feldamplituden interferieren; eine nachfolgende Leistungsmessung ist nicht ohne Weiteres eine Messung eines vorzeichenbehafteten Realwerts. In inkohärenten Architekturen werden gewichtete Intensitäten beziehungsweise Photoströme summiert. Positive und negative Gewichte benötigen eine explizite Codierung oder differenzielle Auslese. Die jeweilige Architektur entscheidet, an welcher Stelle die Summation erfolgt.

Tabelle 2. Allgemeiner hybrider Signalweg

Schritt	Operation	Zu prüfende Systemgröße
1. Elektronische Eingabe	Schlüssel lesen, skalieren und packen	Speicher und Hostzeit
2. Elektrooptische Codierung	Werte in Phase / Amplitude abbilden	Auflösung, Modulation und Energie
3. Photonischer Kern	Lineare Transformation / bestätigte Nichtlinearität	Kennlinie, Verlust und Störung
4. Auslese	Detektion, Wandlung und Übertragung	Rauschen, Raster und Schnittstelle
5. Elektronische Ausgabe	Entscheiden, tauschen / akkumulieren, validieren	Vollständige Sortierqualität und Zeit

Eigene schematische Darstellung; kein Q.ANT-Layout. Bei einer bestätigten Kaskade können interne Grenzen anders liegen.

Die Abbildung zeigt einen allgemeinen hybriden Signalweg, keine Rekonstruktion des Q.ANT-Chips. Ein optischer Kern arbeitet nicht isoliert: Eingaben werden vorbereitet, gewandelt, verarbeitet,

ausgelesen und elektronisch weiterverwendet. Speicher und Steuerung bleiben Teil der Systemgrenze. [1, 12, 22]

2.3 Interferenz als Beispiel einer periodischen Kennlinie

Eine Änderung des Brechungsindex verändert die optische Phase. Für einen Weg der Länge L und Wellenlänge λ gilt im einfachen Modell:

$$\Delta\varphi = \frac{2\pi L\,\Delta n}{\lambda}$$

Für einen idealen symmetrischen und verlustlosen Interferometer ergibt sich an einem Ausgang:

$$P_{\text{out}} = \frac{P_{\text{in}}}{2}\left[1 + \cos(\Delta\varphi)\right]$$

Eine Photodiode liefert im linearen Betriebsbereich I = ℛP mit der Responsivität ℛ. Die gemessene Leistung ist nicht negativ. Eine Differenz zweier positiver Photoströme kann dagegen ein vorzeichenbehaftetes Signal erzeugen. Die periodische Leistungskennlinie erklärt, warum eine nichtlineare Funktion physikalisch zugänglich sein kann. Sie ist aber weder automatisch ReLU noch eine robuste Schwellwertentscheidung.

Lithiumniobat erlaubt schnelle elektrooptische Phasenmodulation. Das in [16] untersuchte TFLN-System ist ein eigenständiger Forschungsaufbau; daraus folgen keine konkreten Q.ANT-Eingangsgrenzen oder frei kaskadierbaren Q.ANT-Operationen. Die native SDK-Funktion tcos ist zudem nicht mit dem idealen Kosinus der obigen Erklärung gleichzusetzen.

2.4 APC als programmierbares Rechenkonzept

Macho-Ortiz et al. beschreiben Analog Programmable-Photonic Computation auf Basis programmierbarer integrierter Photonik. Die elementare Informationseinheit, das Anbit, verwendet zwei komplexe Amplituden. Lineare Transformationen werden in Klassen programmierbarer Gatter eingeordnet; Rückkopplung erweitert kombinatorische zu sequenziellen Systemen. [2, insbesondere Abschnitte zu Anbits und Gattern]

Die Ähnlichkeit einer zweikomponentigen Darstellung mit Quantenformalismen macht APC nicht zu einem Quantencomputer. Die hier genutzten Signale sind klassische optische Größen. Ebenso folgt aus einem allgemeinen APC-Gatterformalismus keine garantierte Umsetzung jedes Sortierbausteins in einem kommerziellen SDK. Für diese Arbeit liefert APC die physikalisch-mathematische Einordnung; die konkrete Implementierung wird an überprüfbaren API-Funktionen festgemacht.

2.5 Verlust, Übersprechen, Rauschen und Präzision

Verlust schwächt das Signal. Ein idealer passiver Pfad mit konstantem Transmissionfaktor a je Stufe hat nach D Stufen die Leistung $P_0 a^D$. Das ist zunächst ein Leistungsmodell. Wie das Signal-Rausch-Verhältnis skaliert, hängt davon ab, wo Rauschen hinzugefügt, mitverstärkt, korreliert oder durch Regeneration verändert wird. Eine universelle lineare oder geometrische SNR-Abnahme wird nicht angenommen.

Kohärentes Übersprechen kann je nach Phase konstruktiv oder destruktiv interferieren; es ist nicht ausschließlich Dämpfung. Inkohärente störende Anteile können einen Hintergrund bilden. Quantisierung beschreibt diskrete Repräsentation und Entscheidungsschwellen. Zufälliges Rauschen, systematische Offsets und Drift sind davon zu unterscheiden. Die Zuverlässigkeitsliteratur motiviert die getrennte Betrachtung dieser Effekte. [1, 21, 30, 33, 34]

Die Begriffe Schlüsselbreite, Speicherformat, Quantisierungsschritt und ENOB bezeichnen verschiedene Größen. Ein BF16-Array belegt ein Softwareformat; es belegt keine 16 effektiven Analogbits. Ein simulierter Schritt Δ ist eine Modellfestlegung; erst eine definierte Messung kann eine physikalische Auflösung oder effektive Bitzahl begründen.

2.6 Regeneration und optische Tiefe

Regeneration stellt ein Datensignal wieder her und begrenzt damit die Länge eines unregenerierten Abschnitts. Eine O/E/O-Grenze kann dies durch Detektion, elektronische Verarbeitung und erneute Emission leisten. Optische Regeneration ist ebenfalls Gegenstand der Forschung, beispielsweise mittels nichtlinearer Signalverarbeitung. Solche Techniken benötigen zusätzliche Komponenten und Betriebsbedingungen. [20]

Elektronische Steuerung allein ist noch keine Regeneration des optischen Nutzsignals. Umgekehrt kann eine hostfreie Ausführung intern bereits Wandlungen und elektronische Speicher verwenden. Für D muss der tatsächliche Datensignalpfad bekannt sein. Aus einer Python-Funktion, einer Zahl von SDK-Aufrufen oder einer Marketingbeschreibung lässt sich D nicht ablesen.

3. Literaturauswertung und Konsequenzen für das Mapping

3.1 Vorgehen und Evidenzarten

Die Recherche verbindet Originalarbeiten zu Sortiernetzwerken, physikalischen Rechenmodellen und hybriden photonischen Systemen. Der bereitgestellte Korpus enthält 25 PDF-Dateien, darunter eine umfangreiche Recherchemappe und eine Sammlung optischer Sortierarbeiten. Diese Sammeldokumente werden nicht als zusätzliche unabhängige Experimente gezählt. Die beiden Präsentationen liefern den thematischen Zusammenhang; technische Aussagen werden an Originalquellen und Code geprüft. Die Korpuszuordnung und ergänzende Literatur stehen im Anhang.

Für die Interpretation werden vier Arten von Aussagen getrennt: mathematische Herleitung, veröffentlichter Geräte- oder Demonstratorbefund, eigene CPU-/Simulationsauswertung und ausdrückliche Annahme. Ein in einer Quelle projizierter Ausbau bleibt eine Projektion. Ein neuronales Genauigkeitsresultat ist kein Nachweis für fehlerfreies Sortieren benachbarter Schlüssel. Eigene Ableitungen werden als solche bezeichnet.

3.2 Systemintegration statt isolierter optischer Operation

Stroev und Berloff [40] ordnen analoge photonische Plattformen insbesondere für Inferenz und Optimierung ein. Diese Übersicht liefert keine pauschale Nullfehlerrate für Sortierung. Die Fehlerfreiheit idealer Rechenmodelle darf nicht als physikalische Eigenschaft übernommen werden.

McMahon [1] erklärt, welche optischen Eigenschaften einen Rechenvorteil ermöglichen können und weshalb kein einzelnes Merkmal diesen garantiert. Daraus folgt für die Arbeit: Nicht nur die Ausbreitungszeit des Lichts betrachten, sondern Datenvorbereitung, erreichbare Präzision und Einbettung in das vollständige Programm.

Ahmed et al. [12] beschreiben einen integrierten hybriden Prozessor mit vier photonischen 128×128-Kernen und digitaler Steuerung. Die in der Quelle angegebene Leistungsverteilung umfasst 43,7 % Logik/NoC/SRAM, 35,6 % Datenwandler, 16,4 % photonische Kerne und 4,2 % PCIe. Diese Demonstratorzahlen veranschaulichen die Bedeutung der Peripherie; sie sind keine Kostenparameter unseres Q.ANT-Sorters. Nichtlinearitäten und Akkumulation werden in dieser Architektur digital eingebettet.

Farmakidis et al. [32] ordnen neuronale Photonik unter anderem nach der Position der optisch-elektronischen Übergänge. Al-Qadasi et al. [33] behandeln Skalierungsgrenzen über optische und elektrische Linkbudgets. Beide Perspektiven begründen die expliziten Ausführungsgrenzen im vorliegenden Mapping: Eine große optische Parallelität ist nur nutzbar, wenn Eingaben, Auslese und Weiterverarbeitung mithalten.

3.3 Hybride Präzision

Meng et al. [13] kombinieren digitale und analoge Verarbeitung, um höhere numerische Präzision zu erreichen. Gong et al. [17] untersuchen in LightMat-HP niederbitrige photonische Teiloperationen und digitale Rekonstruktion für GEMM. Giamougiannis et al. [34] verbinden begrenzte Kernabmessungen mit Tiling und einer an die Aufgabe angepassten Präzision.

Die gemeinsame Folgerung ist keine kostenlose Genauigkeitssteigerung: Zerlegung, Wiederholungen, Akkumulation und zusätzliche Konversion müssen mitgezählt werden. Für Sortierung ist besonders der kleinste relevante Schlüsselabstand kritisch. Ein kleiner mittlerer Matrixfehler kann eine Vergleichsentscheidung genau an einer Grenze dennoch kippen. Daher werden hier Gleichstände und

benachbarte Schlüssel ausdrücklich getestet, statt nur einen globalen mittleren Rechenfehler zu messen.

3.4 Kalibrierung und selektive digitale Verarbeitung

Zhao et al. [14] kombinieren eine Offline-Voreinstellung mit Online-Anpassung für programmierbare optische Prozessoren. Der HOOC-Ansatz zeigt, dass Konfigurationsfehler und spätere Korrektur eigene Arbeitsschritte sind. Eine gemessene Übertragungsfunktion kann ohne Kalibrierung nicht als dauerhaft exakt gelten.

HyAtten [15] untersucht die selektive Weiterverarbeitung problematischer Werte in einem digitalen Pfad, um hochauflösende Wandlung nicht für jeden Wert zu erzwingen. Das motiviert eine mögliche spätere Sortiervariante: Unsichere Vergleiche könnten elektronisch geprüft werden. Diese Variante ist im aktuellen Sortercode jedoch nicht implementiert. Ihre Nutzenbewertung müsste Erkennungskosten, falsch übersehene Grenzfälle und den Anteil digitaler Nacharbeit einbeziehen. Eine nachträgliche Reparatursortierung wird in den vorliegenden Versuchen nicht eingesetzt.

3.5 Native Nichtlinearität und optische Wandlung

Sakata [18] untersucht eine photonikbasierte Analog-Digital-Konversion mit nichtlinearen Fabry-Perot-Resonatoren. Nauman et al. [19] behandeln eine photonische ADC-Architektur zur Unterstützung von MAC-Operationen. Die Recherche zu PCM-basierten Wandlern [31] adressiert ebenfalls die Schnittstelle zwischen Darstellung und Verarbeitung. Diese Arbeiten zeigen alternative technische Ansätze; ihre Auflösung, Energie- oder Schwellenwerte werden nicht auf Q.ANT übertragen.

Hu et al. [16] demonstrieren TFLN-basierte Rechenoperationen mit elektrooptischer Modulation; die beschriebenen Anwendungen enthalten auch elektronische Aktivierungen. Xu et al. [20] behandeln optische Signalverarbeitung und Regeneration in einem breiten Übersichtsrahmen. Zhou et al. [35] untersuchen quantenemitterbasierte nichtlineare Aktivierungen in einer anderen Architektur. Keiner dieser Befunde belegt die hostfreie Kaskadierbarkeit der im vorliegenden SDK benutzten Differenz- und Periodizitätsfunktionen.

3.6 Rauschmodellierung als strukturgebender Beitrag

López-March et al. [21] modellieren physikalische Störquellen zunächst als Photostromschwankungen und propagieren sie über den Empfänger in die APC-Darstellung. Die Quelle trennt unter anderem thermisches Rauschen, Schrotrauschen und relative Intensitätsschwankungen. Die gemessene Silizium-PIP-Anordnung ist nicht unser Q.ANT-System. Übernommen werden daher die Trennung der Störstellen, Einheiten und Fehlerfortpflanzung, nicht ein vermeintlich universelles σ.

Für unsere Vergleiche müssen ein Signalabstand und die zugehörige Störung in derselben Einheit vorliegen. Die Aussage „gleiche Rauschstärke“ ist ohne Schnittstelle, Normierung und Quantisierung unvollständig. Das gemeinsame Modell in Kapitel 9 setzt diese Konsequenz unmittelbar um.

3.7 Energieeffizienz: was sich ableiten lässt

Bahr, Steinmeyer und Pernice [22] analysieren Konversion, Gewichtseinstellung, Speicherzugriffe und optische Verluste auf Systemebene. Große dichte Matrixoperationen können Wandlungskosten auf viele MACs verteilen. Das untersuchte Kostenmodell schließt digitale Steuerlogik jedoch ausdrücklich aus; es ist daher keine vollständige Vermessung unseres Sortierprogramms. Gewichtswiederverwendung, optisches Verlustverhalten und Datenversorgung sind wesentliche Voraussetzungen.

Die Vergleichstabelle der Quelle enthält auch einen B100-Referenzpunkt; die pauschale Aussage, es würden ausschließlich alte digitale Systeme berücksichtigt, trifft damit auf diese Fassung nicht zu. Die entscheidende Grenze bleibt: Peak- oder Literaturwerte bei bestimmten Präzisionen ersetzen keinen aufgabengleichen CPU/GPU-Sortierbenchmark. Ein dünner Zweieingangs-Differenzkernel kann die Amortisierung einer großen dichten MVM nicht automatisch nutzen. [22, Abschnitt III, Tabelle II und Tabelle V]

Für die Masterarbeit folgt daraus eine prüfbare Reihenfolge: funktionales Mapping, numerische Zuverlässigkeit, vollständige Laufzeit und erst anschließend Energie unter derselben Systemgrenze. Ohne gemessene Leistungsaufnahme wird kein Energiegewinn aus einer Hersteller-GOPS-Zahl berechnet.

3.8 Warum PRISM eine andere Zielaufgabe hat

PRISM [23] berechnet photonische Ähnlichkeitsscores zur Auswahl von KV-Cache-Blöcken und verbindet diese mit elektronischer Top-k-Auswahl. Entscheidend ist der Nutzen einer näherungsweisen Vorauswahl im Inferenzablauf, nicht eine vollständige stabile Sortierung aller Werte. Die Arbeit wird deshalb als verwandter Ansatz für eine tolerantere Auswahlaufgabe behandelt.

Recall@k ergänzt unsere Auswertung, weil es die Folgen kleiner Rangfehler für eine Auswahl sichtbar macht. Es macht die Versuche aber nicht zu einem PRISM-Benchmark: Es fehlen dessen Eingaben, Scoreverteilung, Speicherzugriffe und nachgelagerte Modellgüte. Ein hoher Recall der größten Integer-Schlüssel allein rechtfertigt keine Aussage über LLM-Inferenz.

4. Komplexitätsmodell C = (T, S, H, D)

4.1 Einheitliche Definitionen

T: gesamte sequenzielle Arbeit einer vollständigen Sortierung. Gezählt werden skalare arithmetische Operationen, Vergleiche, logische Entscheidungen, Speicherzugriffe und notwendige Datensatzbewegungen mit konstanten Kosten je Modellschritt. Exakte Realwertarithmetik ist eine Idealisierung. Eingabe, Initialisierung, Akkumulation, Gültigkeitsprüfung und Ausgabe gehören dazu; eine parallele Stufe wird in ihre skalaren Operationen aufgelöst.

S: maximal gleichzeitig benötigter logischer Speicher für einen Batch. Gezählt werden Daten und Hilfszustand in logischen Wörtern. Ein Schlüssel, Index oder Zähler gilt im Modell als Wort. Das ist keine Bitkomplexität. Bei einer endlichen Implementierung haben diese Objekte konkrete Formate und Bytekosten; diese werden separat dokumentiert.

H: installierte Verarbeitungspositionen auf Wortebene. Die Granularität bleibt zwischen den Architekturzeilen fest. H ist weder die Zahl von Python-Aufrufen noch vollständige Chipfläche. Speicherzellen, Leitungsgeometrie, Optik, Treiber und Wandler benötigen für eine Flächenbewertung ein zusätzliches physisches Layoutmodell.

D: maximale Zahl aufeinanderfolgender optischer Stufen ohne Regeneration des Datensignals. D endet an einer tatsächlichen Wiederherstellung des Nutzsignals. Mehrfachverwendung einer regenerierenden Schleife erhöht nicht automatisch die Tiefe eines einzelnen unregenerierten Abschnitts.

Das Tupel bleibt unverändert. Sekundenzahlen, Durchsatz, Energie, Signalabstände, Fehlerquoten und Aufrufzahlen sind ergänzende Evaluierungsgrößen, keine neuen Komplexitätskoordinaten.

4.2 Warum parallele Architekturen dasselbe T haben können

Eine Bitonic-Stufe führt N/2 unabhängige Compare-and-Exchange-Operationen aus. In einer räumlichen Pipeline können diese gleichzeitig stattfinden; im real-RAM-artigen T tragen sie trotzdem $\Theta(N)$ Arbeit bei. Sowohl Pipeline als auch Rezirkulation benötigen voneinander abhängige Stufen. „Pipeline = sequenziell, Rezirkulation = vollständig parallel“ wäre deshalb eine falsche Gegenüberstellung.

Die Unterschiede liegen unter anderem in installierter Hardware, Überlappung mehrerer Batches, Serialisierung, Auslastung und physischer Taktung. Gleiche asymptotische Arbeit bedeutet weder gleiche Latenz noch gleichen Durchsatz. Ein Pipeline-Ergebnisintervall kann kürzer als ihre Zeit bis zum ersten vollständigen Ergebnis sein.

4.3 Historische Modelle als Bezugspunkte

Thompson [3] beschreibt physische Fläche und Zeit in einem VLSI-Modell und leitet für die DFT unter seinen Annahmen eine Schranke $AT^2 \geq N^2/16$ ab. Die Einsicht betrifft räumliche Ressourcen und Kommunikation. Sie ist keine Sortieruntergrenze und A ist nicht mit unserem S oder H identisch.

Caulfield [4] bezieht normalisierte räumliche, zeitliche und Fan-in-Ressourcen auf die Problemkomplexität. Die Relation $X_A X_T X_F \geq X_P$ erklärt, warum eine breite optische Reduktion nicht allein anhand der Zahl abstrakter Stufen bewertet werden kann. Die zeitliche Größe dieser Quelle ist nicht unser sequenzielles T.

Li und Monticone [5] verbinden die Struktur linearer Operatoren mit physischem Raumbedarf und Kommunikation. Das motiviert die Beachtung von Nichtlokalität und Routing. Ein Ergebnis für einen linearen Operator lässt sich jedoch nicht unverändert auf die nichtlineare Gesamtsortierung

anwenden. Das verwendete Tupel ist eine operationalisierte Vergleichsbasis, kein aus den drei Arbeiten unverändert übernommenes Gesetz.

4.4 Untere, obere und enge Schranken

Ω bezeichnet eine asymptotische untere, O eine obere und Θ eine passende untere und obere Schranke. Diese Notationen bedeuten nicht Best-, Worst- und Average-Case. Die folgenden Tabellen betreffen genau die spezifizierten Architekturen und Repräsentationen, nicht das optimale Sortieren über alle denkbaren Verfahren. Für $N \geq 2$ sind alle Logarithmen zur Basis zwei.

Tabelle 3. Untere Schranken der spezifizierten Architekturen

Architektur	T	S	H	D
Pipeline-Bitonic	$\Omega(N \log^2 N)$	$\Omega(N)$	$\Omega(N \log^2 N)$	$\Omega(\log^2 N)$ / $\Omega(1)$
Beyette, rezirkulierend	$\Omega(N \log^2 N)$	$\Omega(N)$	$\Omega(N)$	$\Omega(1)$
Desmulliez, Shuffle	$\Omega(N \log^2 N)$	$\Omega(N)$	$\Omega(N)$	$\Omega(1)$
Louri, materialisierte Ränge	$\Omega(N^2)$	$\Omega(N^2)$	$\Omega(N^2)$	$\Omega(1)$
Radix, feste Parameter	$\Omega(N)$	$\Omega(N)$	$\Omega(1)$	$\Omega(1)$

Eigene Ableitung unter den Definitionen und Annahmen in Kapitel 4. D der Pipeline: passiv / nach jeder Stufe regeneriert.

Tabelle 4. Enge Schranken der spezifizierten Architekturen

Architektur	T	S	H	D
Pipeline-Bitonic	$\Theta(N \log^2 N)$	$\Theta(N)$	$\Theta(N \log^2 N)$	$\Theta(\log^2 N)$ / $\Theta(1)$
Beyette, rezirkulierend	$\Theta(N \log^2 N)$	$\Theta(N)$	$\Theta(N)$	$\Theta(1)$
Desmulliez, Shuffle	$\Theta(N \log^2 N)$	$\Theta(N)$	$\Theta(N)$	$\Theta(1)$
Louri, materialisierte Ränge	$\Theta(N^2)$	$\Theta(N^2)$	$\Theta(N^2)$	$\Theta(1)$
Radix, feste Parameter	$\Theta(N)$	$\Theta(N)$	$\Theta(1)$	$\Theta(1)$

Eigene Ableitung unter den Definitionen und Annahmen in Kapitel 4. D der Pipeline: passiv / nach jeder Stufe regeneriert.

Tabelle 5. Obere Schranken der spezifizierten Architekturen

Architektur	T	S	H	D
Pipeline-Bitonic	$O(N \log^2 N)$	$O(N)$	$O(N \log^2 N)$	$O(\log^2 N)$ / $O(1)$
Beyette, rezirkulierend	$O(N \log^2 N)$	$O(N)$	$O(N)$	$O(1)$
Desmulliez, Shuffle	$O(N \log^2 N)$	$O(N)$	$O(N)$	$O(1)$
Louri, materialisierte Ränge	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$
Radix, feste Parameter	$O(N)$	$O(N)$	$O(1)$	$O(1)$

Eigene Ableitung unter den Definitionen und Annahmen in Kapitel 4. D der Pipeline: passiv / nach jeder Stufe regeneriert.

Bei der Pipeline bezeichnet die erste D-Alternative passive Weiterleitung, die zweite Regeneration nach jeder Stufe. Bei Rangsortierung gilt S für die materialisierte Matrixdarstellung. Bei Radix bleiben

Schlüsselbreite, Radix, Passzahl und Zahl der Verarbeitungsströme konstant; H schließt wachsenden Datenspeicher aus. Diese Annahmen sind Teil der jeweiligen Schranke.

4.5 Herleitung der fünf Zeilen

Für $N = 2^m$ hat das kompakte Bitonic-Netzwerk die Stufenzahl K und Vergleichszahl C_B:

$$K = \sum_{j=1}^{m} j = \frac{m(m+1)}{2}, \qquad C_B = \frac{NK}{2}$$

Jede Vergleichsoperation erfordert eine positive konstante Zahl skalarer Schritte. Der vollständige feste Plan liefert daher sowohl Ω(NK) als auch O(NK). Eingabe und Ausgabe fügen Θ(N) hinzu. Der feste Shuffle-Plan benötigt $P = m^2 - m + 1$ Durchläufe einschließlich Bypass; auch das Bewegen von N Wörtern in jedem dieser Durchläufe ändert die Ordnung $\Theta(N \log^2 N)$ nicht. [6-9]

Ein einzelner lebender Batch und eine konstante Zahl wiederverwendeter Puffer begründen S = Θ(N) für die modellierten Bitonic-Ausführungen. Ein räumlicher Aufbau installiert Θ(NK) Vergleichspositionen; die wiederverwendeten Arrays installieren Θ(N). Passiv kaskadierte Nutzdaten durchqueren Θ(K) Stufen. Regenerierende Übergänge begrenzen D auf eine konstante Segmentlänge.

Die materialisierte Rangarchitektur verarbeitet eine quadratische Vergleichsebene. Broadcasts, Unterschiede, Schwellen, Rangsumme und Auswahl erfordern im skalaren Modell eine konstante Zahl quadratischer Durchläufe. Daraus folgen T, S und H jeweils $\Theta(N^2)$. Eine feste Anzahl breiter optischer Ebenen hat D = Θ(1), ohne dass Lichtleistung oder Fläche konstant wären. Eine gestreamte elektronische Rangimplementierung könnte weniger Speicher benötigen; sie ist nicht die hier definierte Matrixrepräsentation. [10]

Stabiles Radixsortieren mit q Durchläufen und R Buckets kostet Θ(q(N+R)). Bei festem Wortformat und Radix reduziert sich dies auf Θ(N). Digitextraktion ist dabei eine ausdrücklich zugelassene konstant teure Integeroperation. Diese Zeile beansprucht kein lineares vergleichsbasiertes Sortieren beliebiger reeller Werte. Feste Verarbeitungsströme liefern H = Θ(1), wachsende Datenspeicher S = Θ(N). [11]

Die Tabellen beschreiben idealisierte Architekturfamilien. Im tatsächlich benutzten Python-Code können vorab materialisierte Paarpläne, Batchvektoren und Zwischenkopien zusätzlichen Speicher belegen. Insbesondere hält der Bitonic-Code den Vergleichsplan bereit; sein tatsächlicher Speicher ist nicht allein durch die idealisierte Ein-Batch-Schranke garantiert. Diese Umsetzungskosten dürfen nicht aus der Tabelle herausgelesen werden.

5. Die fünf untersuchten Sortierarchitekturen

5.1 Räumliches Pipeline-Bitonic nach Stirk und Athale

Die Architektur ordnet Compare-and-Exchange-Module in aufeinanderfolgenden Stufen an und verbindet sie durch feste optische Kanalumordnungen. Ein bitonischer Merge kombiniert eine aufsteigende und eine absteigende Teilfolge. Die Merge-Tiefen 1 bis m ergeben den vollständigen Sorter. Innerhalb jeder Stufe sind die Paare unabhängig; zwischen den Stufen besteht Datenabhängigkeit. [7, Abschnitt II]

Das Paper untersucht verschiedene Technologien für Vergleich, Verriegelung und Austausch. Entscheidend für D ist die Unterscheidung zwischen regenerierender Weitergabe und passiver Führung des Nutzsignals. Eine optische Verbindungsstruktur allein belegt keine störungsfreie Kaskade. Analoge Schwankungen, Dynamikbereich und Schaltschwellen können die nutzbare Tiefe begrenzen. [7, insbesondere Abb. 5-6]

Für die moderne Implementierung wird die logische Folge aktiver C&E-Stufen übernommen. Ganzzahlige Datensätze werden dabei nicht als originale bitserielle Lichtfolgen dargestellt. Der Softwareplan sammelt Paare, berechnet Unterschiede und vertauscht die Originaldatensätze elektronisch. Die Quelle begründet die Struktur, nicht die Geschwindigkeit oder Energie dieser Adaption.

5.2 Rezirkulierendes Bitonic nach Beyette et al.

Beyette et al. verwenden zwei Smart-Pixel-Arrays wiederholt. Daten gelangen optisch zwischen den Arrays; elektronische Vergleichslogik, Verriegelung und Synchronisation unterstützen die Verarbeitung. Die wiederholte Nutzung reduziert installierte Hardware gegenüber einer vollständig ausgelegten Pipeline. Dafür kann die Pipeline mehrere Batches stärker überlappen. [8, Abschnitte 2 und 3]

Die Quelle unterscheidet Systemkapazität, Antwortzeit und Ergebnisintervall. In ihrem Vergleich mit bitserieller Ein-/Ausgabe ergibt sich für beide vollständigen Batch-Durchläufe $P + b - 1$ Taktzyklen unter den dortigen Annahmen; die Ausgabeintervalle betragen b für die Pipeline und P für Rezirkulation. Diese konkrete Aussage darf nicht zu einer universellen Gleichheit sämtlicher physischer Laufzeiten erweitert werden. [8, Abschnitt 3.C]

Für $N = 16$ ist $m = 4$ und $P = 13$. Bei $b = 8$ ergibt sich in diesem Quellenmodell eine vollständige Antwort nach 20 Takten. Die elektronische Wiederherstellung zwischen Übertragungen begrenzt D des einzelnen optischen Segments. Mehr Schleifendurchläufe erhöhen Arbeit und Verkehr, nicht automatisch die unregenerierte Tiefe.

5.3 Perfect-Shuffle-Bitonic nach Desmulliez et al.

Diese Architektur verwendet ein wiederverwendetes Sorting-Node-Array, Speicher und konfigurierbare Kontrollmasken. Ein Perfect Shuffle verändert die Zuordnung der Kanäle. Nodes vergleichen, tauschen oder lassen Daten passieren. Bitserielle Verarbeitung startet beim höchstwertigen Bit; zusätzliche Zyklen übernehmen Maskenladen und Reset. [9, Abschnitte 2-5]

Der feste Shuffle-Plan hat $P = m^2 - m + 1$ Schritte. Die im Paper beschriebene Zyklusrechnung verwendet $(b + 2)P$, also für $N = 16$ und $b = 8$ insgesamt 130 Zyklen. Sie zählt andere Hardwareabläufe als die kompakte aktive Bitonic-Implementierung. Gleiches asymptotisches T bedeutet deshalb nicht, dass diese beiden Quellen dieselbe Taktzahl vorhersagen.

Die Architektur ist besonders aufschlussreich für den Zielkonflikt zwischen nichtlokaler optischer Verbindung und elektronischer Zustandsverwaltung. Im heutigen Funktionsmapping werden keine separaten Desmulliez-Sortergebnisse als unabhängige photonische Messreihe ausgegeben. Das frühere softwareseitige Ausführen desselben Bitonic-Ordnungsplans unter verschiedenen Architekturnamen wäre kein eigenständiger Hardwarevergleich.

5.4 Broadcast-and-Compare nach Louri et al.

Louri et al. verteilen die Eingaben in eine breite Vergleichsstruktur. Paarweise Ordnungsergebnisse werden zu Rängen zusammengeführt; aus diesen folgen die Ausgabepositionen. Eine zusätzliche Behandlung gleicher Schlüssel macht die Rangzuweisung eindeutig. Smart Pixels enthalten dabei auch elektronische Verarbeitung. Die populäre Beschreibung als vollständig optische Differenzentscheidung wäre zu weitgehend. [10, Gleichungen 3-7 und Abschnitt 3]

Die konstante Zahl breiter Quelloperationen setzt eine mit N wachsende Vergleichsfläche und geeignete breite Reduktionen voraus. Im skalaren real-RAM-artigen Modell expandiert diese Arbeit zu $\Theta(N^2)$. Für das Quellbeispiel [7, 8a, 2, 8b, 5] ergeben sich die einsbasierten Ränge [3, 4, 1, 5, 2] und die stabile Ausgabe [2, 5, 7, 8a, 8b]. Unsere nullbasierte Zählweise beschreibt dieselbe ideale Ordnung.

Historische Timingrechnungen in den Projektunterlagen reproduzieren eine Summe von 60,2 ns für einen vollständigen Zyklus und 36,8 ns als Periode. Sie sind keine NPU-Messung. Zudem ergeben die im Quellenkontext genannten 10 pJ und 1,5 mW über E/P etwa 6,67 ns, während der Text ungefähr 13,33 ns verwendet. Diese Diskrepanz bleibt offen; sie wird nicht durch eine präzise aussehende moderne Laufzeit ersetzt.

5.5 In-Network-Radix nach Mizutani et al.

Mizutani et al. kombinieren elektronische Sortierkerne und HBM auf mehreren FPGAs mit optischen Speicher-zu-Speicher-Verbindungen. Der 32-Bit-Schlüssel wird in acht 4-Bit-Counting-Sort-Pässen verarbeitet. Stabile Verteilung, Speicherverkehr und Netzwerkstruktur stehen im Vordergrund. Die Photonik stellt Kommunikation bereit; sie berechnet nicht die Vergleichsentscheidung eines analogen C&E. [11]

Die Quelle berichtet für den Acht-FPGA-Aufbau 37,2 GB/s bei 16 GiB Daten und 9,2 MB/J unter den beschriebenen Bedingungen. Die Skalierung auf 256 FPGAs mit 983 GB/s bei 512 GiB ist eine modellgestützte Ausbauanalyse, kein gleichartiger gemessener 256-FPGA-Demonstrator. Die unterschiedlichen Evidenzarten bleiben getrennt.

Diese Architektur ist im Gesamtvergleich wichtig, weil sie zeigt, dass ein photonischer Systembeitrag auch über Datenbewegung statt analoge Arithmetik entstehen kann. Ihre großen Datenmengen, Integeroperationen und Hardwareplattform sind jedoch nicht mit unseren kleinen CPU-Listen gleichzusetzen. Sie liefert einen Architekturkontrast, keine direkt einsetzbare Leistungsbaseline.

5.6 Auswahl für die moderne Evaluation

Tabelle 6. Rolle der fünf Architekturen in dieser Arbeit

Architektur	Übernommene Idee	Umsetzung im aktuellen Versuch
Stirk / Athale	Feste aktive C&E-Stufen	Bitonic-Funktionsmapping
Beyette	Wiederverwendete Vergleichsfläche	Ressourcen- und Latenzkontrast
Desmulliez	Shuffle, Masken und Bypass	Quellenbasierte Zählung, kein eigener NPU-Lauf
Louri	Breite Paarphase und Ränge	Rang-Funktionsmapping
Mizutani	Photonische Datenbewegung	Hybrider Architekturkontrast

Die beiden Kernmappings isolieren den Gegensatz zwischen schmalen abhängigen Stufen und einer breiten Paarphase. Die übrigen Architekturen begründen die Einordnung von Hardwarewiederverwendung, Routing und Kommunikationskosten. So bleiben Literaturvergleich und ausführbare Evaluierung verbunden, ohne fünf nicht vorhandene Geräteimplementierungen zu suggerieren.

6. Mathematische Mappings und stabile Datensätze

6.1 Die Aufgabe präzise definieren

Eine Eingabeliste besteht aus N Originaldatensätzen mit ganzzahligem Schlüssel x_i und Originalindex i. Die vollständige Ausgabe muss jeden Datensatz genau einmal enthalten und die Schlüssel aufsteigend ordnen. Bei gleichen Schlüsseln müssen die Originalindizes aufsteigend bleiben. Die ideale Referenz ist damit die lexikografische Ordnung (x_i, i).

Das analoge Modell liefert nur ein Vergleichssignal. Schlüsselwerte und Indizes bleiben als unveränderte Originaldaten gespeichert. Diese Trennung ist entscheidend: Ein korrektes Vorzeichen genügt zur Datensatzentscheidung, eine verzerrte Differenz reicht dagegen nicht zur exakten Rekonstruktion von Minimum und Maximum.

6.2 Ein linearer Unterschied und eine nichtlineare Entscheidung

Für B unabhängige Paare wird X als B×2-Matrix angelegt. Die Gewichtsmatrix hat Form 1×2:

$$X_{p,:} = (a_p, b_p), \qquad W = (1, -1), \qquad Y = XW^{\mathsf{T}}$$

Jede Zeile liefert $d_p = a_p - b_p$. Das ist linear. Die Entscheidung, welches Original nach links oder rechts beziehungsweise auf welchen Rang gehört, ist nichtlinear. Eine reine MVM implementiert deshalb noch keine vollständige Sortierung.

Für exakte, unbeschnittene Werte gilt außerdem r = max(d, 0), min(a,b) = a − r und max(a,b) = b + r. Diese ReLU-Identität erklärt eine mögliche Zerlegung, nicht ihren Ausführungsort. Im untersuchten SDK liegt ReLU auf dem Host. [36]

Ein Gegenbeispiel verhindert eine falsche Verwendung des gesättigten Komparators: Bei 8 Bit ist d für (255,0) gleich 255/256. Die signorientierte Referenz kann dies auf 127/256 beschneiden. Das Vorzeichen bleibt korrekt, die ReLU-Rekonstruktion würde aber die Werte 128 und 127 statt 0 und 255 erzeugen. Der Sorter verwendet daher ausschließlich Originaldatensätze zum Austausch.

6.3 Bitonic: Vergleichsplan und Austausch

Der kompakte Batcher-Plan verdoppelt die Gruppengröße von 2 bis N. Innerhalb einer Gruppe halbiert sich die Partnerdistanz. Ein Partner j = i XOR distance bestimmt die jeweilige Verbindung; i < j vermeidet doppelte Paarbearbeitung. Die Gruppe legt aufsteigende beziehungsweise absteigende Richtung fest. Der aktuelle Runner verwendet Zweierpotenzen und lehnt andere Längen für Bitonic ab.

Tabelle 7. Dieselbe Vergleichsfunktion in zwei vollständigen Abläufen

Phase	Bitonic	Rangsortierung
Paare vorbereiten	N/2 Paare der aktuellen Stufe	Alle N(N−1)/2 ungeordneten Paare
Differenz und Entscheidung	Parallel innerhalb einer Stufe	Unabhängige breite Paarphase
Elektronisch weiterarbeiten	Originaldatensätze tauschen	Ränge summieren, Belegung prüfen
Nächster Schritt	Nächste abhängige Stufe bis K	Einmalige Ausgabe aus gültigen Rängen

Die Unterschiede betreffen Abhängigkeiten, Ressourcen und Datenbewegung; beide Mappings verwenden denselben stabilen Ausgabevertrag.

Der Bitonic-Datenweg führt pro Stufe vom elektronischen Sammeln der Paare zum Vergleichssignal und zurück zum elektronischen Austausch. Erst dessen Ergebnis kann in die nächste Stufe eingehen. Ein Batch kann viele unabhängige Listen gruppieren; dadurch verschwinden diese Abhängigkeiten innerhalb einer Liste nicht.

Bei gemessenem Signal đ ordnet die Entscheidung a hinter b, falls đ > 0 oder bei đ = 0 der Originalindex von a größer ist. Der Vergleich verwendet die reisenden Originalindizes, nicht die momentanen Drahtpositionen. Absteigende Netzwerkstufen kehren dieselbe totale Vergleichsordnung um. Ohne Störung resultiert dadurch eine stabile Gesamtausgabe.

Das vollständige Beispiel [10,3,9,1] mit 8-Bit-Normierung:

Tabelle 8. Bitonic-Beispiel für [10,3,9,1]

Stufe	Paare und Richtung	Differenzen	Zustand danach
1	(0,1) auf; (2,3) ab	7/256; 8/256	[3,10,9,1]
2	(0,2) auf; (1,3) auf	−6/256; 9/256	[3,1,9,10]
3	(0,1) auf; (2,3) auf	2/256; −1/256	[1,3,9,10]

Eigene Darstellung auf Grundlage der im Abschnitt angegebenen Herleitung oder Quellen.

Die Originalindexfolge am Ende lautet [3,1,2,0]. Selbst bei fehlerhaften Entscheidungen bleibt das Ergebnis durch elektronische Vertauschungen eine Permutation. Es kann trotzdem falsch geordnet oder instabil sein.

6.4 Rangsortierung: zählen, prüfen, platzieren

Der nullbasierte Rang r_i ist die Zahl der Datensätze, die vor Datensatz i liegen müssen. Ohne Fehler gilt:

$$r_i = \sum_{j \neq i} \mathbf{1}\{x_j < x_i \text{ oder } (x_j = x_i \text{ und } j < i)\}$$

Im Modell ersetzt das gemessene Differenzsignal die Kenntnis von $x_j < x_i$. Für jedes Paar i < j wird nur eine Entscheidung erzeugt: Positives Signal erhöht r_i; negatives oder null erhöht r_j. Null begünstigt also den früheren Originalindex. Die Gegenseite wird komplementär behandelt. Dies spart doppelte Auswertung, garantiert unter Rauschen aber keine Transitivität.

Für [10,3,9,1] ergeben die sechs Paare nacheinander:

Tabelle 9. Stabile Rangbildung für [10,3,9,1]

Paar	Schlüssel	Erhöhter Rang	Ränge nach dem Schritt
(0,1)	10,3	r_0	[1,0,0,0]
(0,2)	10,9	r_0	[2,0,0,0]
(0,3)	10,1	r_0	[3,0,0,0]
(1,2)	3,9	r_2	[3,0,1,0]
(1,3)	3,1	r_1	[3,1,1,0]
(2,3)	9,1	r_2	[3,1,2,0]

Eigene Darstellung auf Grundlage der im Abschnitt angegebenen Herleitung oder Quellen.

Die sequentielle Tabelle dient der Erklärung. Die sechs Unterschiede sind unabhängig und können gemeinsam berechnet werden; die Rangsummen und Platzierung erfolgen danach. out[r_i] = x_i ergibt [1,3,9,10]. Für [3_0,1_1,3_2,1_3] lauten die Ränge [2,0,3,1], sodass [1_1,1_3,3_0,3_2] entsteht.

Unter Rauschen kann beispielsweise a > b, b > c und c > a entschieden werden. Dann kollidieren die Ränge. Der Code prüft, ob jede Position 0 bis N−1 genau einmal belegt ist, und markiert unzulässige Ergebnisse. Diese Prüfung nutzt Belegungszähler; sie sortiert die Ränge nicht nachträglich. Ungültige Rangfälle werden nicht durch einen digitalen Ersatzsorter repariert.

6.5 Strukturprüfung und Reichweite

Der Bitonic-Plan wird auf disjunkte Paarabdeckung je Stufe und passende Gesamtzahl geprüft. Kleine binäre Eingaben bei N = 2, 4 und 8 sowie gespeicherte Ganzzahlkorpora prüfen Ordnung und Stabilität. Die Rangadaption wird anhand von Handbeispielen, Gleichständen und widersprüchlichen Entscheidungen geprüft. Die Quelle von Louri und die Implementierung verwenden teils unterschiedliche Matrixorientierungen und eins- beziehungsweise nullbasierte Ränge; übereinstimmen muss die resultierende Ordnung.

Diese Prüfungen validieren Funktion und Operationsstruktur. Sie bestimmen weder physische MVM-Kapazität noch optische Fan-out-Verluste, Taktgrenzen oder Energiewerte. Nichtlokale optische Verbindungen werden im Funktionsmapping durch elektronische Indizierung ersetzt.

6.6 Vollständige Louri-Matrix als Gleichungsreferenz

Die separate Implementierung louri_reference.py bildet die Matrizen der Quelle direkt ab. Zeile i bezeichnet den konkurrierenden Datensatz, Spalte j den Datensatz, dessen Rang bestimmt wird. Mit Δ = 2^{-b} werden sowohl die ganzzahligen Schlüssel als auch der Einheitsbias normiert:

$$D_{ij} = \Delta(x_j - x_i), \qquad U_{ij} = \mathbf{1}[i > j], \qquad D'_{ij} = D_{ij} - \Delta U_{ij}$$

$$R'_{ij} = \mathbf{1}[D'_{ij} \geq 0], \qquad r_j^{(1)} = \sum_i R'_{ij}, \qquad r_j^{(0)} = r_j^{(1)} - 1$$

Die Diagonale liefert jeweils eine Eins; deshalb beginnt der Quellenrang bei 1. Das veröffentlichte Beispiel [7,8,2,8,5] ergibt [3,4,1,5,2] und eine stabile Reihenfolge der beiden Achten. Zwei Textstellen auf Seite 3089 beschreiben die Gleichstandsrichtung beziehungsweise die betroffene Dreieckshälfte widersprüchlich. Die Referenz folgt den dargestellten Gleichungen und Beispielmatrizen. [10, Gleichungen 3-7]

Ohne Rauschen stimmen die rohe Matrix, die Matrix mit Quantisierung und die bestehende Paaradaption mit stabiler digitaler Sortierung überein: 256 geordnete 4-Bit-Paare, 65.536 geordnete 8-Bit-Paare, sämtliche 65.536 Viererlisten über 4-Bit-Schlüsseln und alle 24.000 gespeicherten Listen. Das sind 155.328 geprüfte Eingaben; überlappende Kontrollmengen sind keine unabhängigen Stichproben. [D8]

6.7 Was der Rauschvergleich tatsächlich prüft

Die Sensitivitätsserie addiert angenommene gaußsche Fehler zu D, vor Bias und Schwelle. Die Standardabweichung beträgt $\sigma = \eta\Delta$, mit η aus {0; 0,05; 0,10; 0,15; 0,20; 0,25}. Untersucht werden unabhängige gerichtete Fehler sowie gekoppelte Fehler $E_{ji} = -E_{ij}$. Die Diagonale ist entweder bekannt und exakt oder separat verrauscht. Quantisierung ist entweder ausgeschaltet, einschließlich Clipping, oder verwendet das bisherige vorzeichenbehaftete Raster vor dem Bias. Bias, Schwelle, Rangsummen und Platzierung bleiben exakt.

Bei gekoppelten Fehlern, exakter Diagonale und diesem Raster stimmen alle gerichteten Entscheidungen mit der Paaradaption überein, auch wenn fehlerhafte Vergleiche Rangkonflikte erzeugen. Dies wurde in 3.600 gepaarten Läufen geprüft. Bei unabhängigen Fehlern können beide Richtungen dagegen gleichzeitig null oder eins liefern. Eine vollständige Matrix ist deshalb ohne Rauschen äquivalent, unter beliebigen Rauschannahmen aber nicht automatisch äquivalent.

Das rohe Quellenmodell besitzt zudem Nullabstände zur Schwelle: Bei [1,0] beträgt $D_{10} = \Delta$ und nach Bias $D'_{10} = 0$. Ein beliebig kleiner darstellbarer negativer Fehler kippt diesen Beitrag. Vorherige Rundung erzeugt hier eine Reserve von einer halben Rasterstufe. Acht Kontrollen mit je 100.000 Paaren bestätigen die zugehörigen gaußschen Wahrscheinlichkeiten innerhalb der festgelegten Stichprobentoleranz. Die vollständige Sensitivitätsserie umfasst 36.000 Auswertungen in 360 Konfigurationen auf den ersten 100 Listen von sechs gespeicherten Datensätzen. [D8]

Tabelle 10. Louri-Referenz und Adaption unter erklärten Rauschannahmen

Rechenweg / Rauschen	Quantisierung	Gültig	Korrekt	Stabil
Ungeordnete Paare	keine	74/100	74/100	1/100
Matrix: gekoppelt	keine	0/100	0/100	0/100
Matrix: unabhängig	keine	0/100	0/100	0/100
Ungeordnete Paare	vor Bias	93/100	88/100	76/100
Matrix: gekoppelt	vor Bias	93/100	88/100	76/100
Matrix: unabhängig	vor Bias	53/100	53/100	53/100

D8: Erste 100 gespeicherte 4-Bit-Listen, N = 16, Duplikate erlaubt; $\sigma = 0{,}25\Delta$. Matrixdiagonale hier exakt. Gekoppelt: $E_{ji} = -E_{ij}$. Keine Hardwaremessung.

Gültigkeit, vollständige Korrektheit und Stabilität werden getrennt ausgewertet. Kendall-τ, Recall@10 und Inversionen nach Schlüsselabstand sind nur bei gültigen Ausgaben definiert; eine zusätzliche Recall-Zahl setzt ungültige Ausgaben ausdrücklich auf null. Die schlechten rohen Matrixwerte beschreiben die gewählte Schwellen- und Rauschannahme, nicht die Zuverlässigkeit eines realen Louri- oder Q.ANT-Geräts. Photodetektoren, optische Summierung, Biasabweichungen, Drift und reale Fehlerkorrelationen sind weiterhin nicht physikalisch charakterisiert.

7. Q.ANT-SDK und periodischer Vergleich

7.1 Überprüfter Softwarestand

Der untersuchte SDK-Stand ist Version 2.3.1, Quellcommit 72a2d99f10240b6df3c6d0f636dfa0e2b5d38902. Die Aussagen dieses Kapitels sind an diesen Stand gebunden; ein späterer SDK-Stand müsste gesondert geprüft werden. Das CPU-Release und seine Herstellerlizenzen liegen im Repository. [36]

Tabelle 11. Geprüfte SDK-Funktionen und ihre Grenzen

Funktion	Im Quellstand sichtbarer Pfad	Aussagegrenze
linear_fprop	Aufruf des Matrix-/Vektortreibers	Dokumentierter API-Pfad; keine gemessene Hardwareleistung
calc_scaled_periodic_nl_fprop	Native periodische Treiberfunktion v·tcos(u)	Reale Kennlinie und Auflösung nicht durch den CPU-Ersatz belegt
relu_fprop	Hostschleife mit max(x,0)	Kein Nachweis nativer optischer ReLU
KAN-Komposition	Treiberaufrufe plus Hostarbeit	Hybrider Ablauf; keine belegte rein optische Kaskade
vollständiges C&E	Kein entsprechender geprüfter primitiver Pfad	Optisches Routing und Kaskadierbarkeit bleiben offen

Eigene Darstellung auf Grundlage der im Abschnitt angegebenen Herleitung oder Quellen.

Das CPU-Backend setzt für die periodische Funktion einen Kosinus ein. Es ist kein kalibrierter digitaler Zwilling des optischen Chips. Der nicht offene native Treiber begrenzt außerdem die Einsicht in interne Puffer, Wandlungen und Scheduling.

7.2 Herleitung des Kandidaten

Für normierte Differenzen $-1 < d < 1$ wird die Phase u auf den monotonen Kosinusast zwischen 0 und π gelegt:

$$u = u_0 - \alpha d, \quad u_0 = \alpha = \frac{\pi}{2}, \quad \cos(u) = \sin(\alpha d)$$

Damit besitzt der ideale Score dasselbe Vorzeichen wie d. Die Konstruktion liefert ein Ordnungssignal, keine exakten Minimum-/Maximum-Werte. Eine reale Funktion f müsste über den gewählten Bereich eine passende monotone, ausreichend aufgelöste Kennlinie besitzen. Periodizität allein genügt nicht.

In endlicher CPU-Präzision wird eine Referenz $y_0 = f_CPU(d = 0)$ abgezogen. BF16 rundet π/2 auf 1,5703125. Das gespeicherte CPU-Ergebnis für die Nullreferenz beträgt 0,000484466552734375. Ohne Korrektur würden gleiche Paare ein positives Signal liefern. Mit $z = y - y_0$ sind alle 256 geordneten 4-Bit-Paare und 65.536 geordneten 8-Bit-Paare in der gespeicherten Prüfung korrekt, einschließlich Gleichständen. [D3]

Tabelle 12. Erschöpfende periodische Paarprüfung im offiziellen CPU-Backend

Schlüssel	Geordnete Paare	Fehler vor Referenz	Fehler nach Referenz	Kleinster Abstand m
4 Bit	256	16	0	0,10155869
8 Bit	65.536	256	0	0,00780869

Quelle: results/periodic/pair_checks.json. Kein Zusatzrauschen; keine Hardwareausführung.

Für das 8-Bit-Paar (10,9) ist die exakte normierte Differenz 1/256. Nach Host-Phasenbildung und BF16-Rundung ist u = 1,5625; der periodische CPU-Ausgang beträgt 0,00830078125. Nach Referenzabzug folgt z = 0,007816314697265625 > 0. Das gleiche Paar (9,9) liefert z = 0. Die diskreten BF16-Phasenschritte erklären, weshalb die kleinste gespeicherte Scorelücke vom glatten Idealmodell abweicht.

7.3 Welche Arbeit der Baustein heute tatsächlich übernimmt

Im vorhandenen Ablauf liegt d nach dem linearen API-Aufruf bereits auf dem Host. Dort können sign(d) und die Gleichstandsregel die Vergleichsentscheidung unmittelbar liefern. Der periodische Folgeaufruf fügt Phasenberechnung, einen weiteren Aufruf, Puffer und abschließende Schwellenentscheidung hinzu. Seine Funktionsfähigkeit ist belegt; ein Nutzen gegenüber der direkten Entscheidung ist in diesem Ablauf nicht nachgewiesen.

Die benötigte Nichtlinearität ist die Ordnungsentscheidung. Auch nach der periodischen Transformation liest die Elektronik das Vorzeichen beziehungsweise einen Toleranzbereich. Der zusätzliche Aufruf vermeidet im aktuellen Code keinen elektronischen Zwischenschritt.

7.4 Affine Phasenbildung: Machbarkeit und Präzisionsgrenze

Die Host-Phasenberechnung kann algebraisch in die lineare Operation verlagert werden. Für die normierten Schlüssel $x = a/2^w$ und $y = b/2^w$ mit Schlüsselbreite w gilt:

$$u = [x, y, 1] \cdot [-\alpha, \alpha, u_0]^{\mathrm{l}}, \quad u_0 = \alpha = \pi/2$$

Die konstante Eingabe eins stellt den Bias bereit. check_affine_periodic.py führt genau diese BF16-MVM und anschließend den periodischen SDK-Aufruf aus. Es berechnet keine Differenz auf dem Host. Allerdings wird die Phase weiterhin als NumPy-Array zurückgegeben und an den zweiten Aufruf übergeben. Die konkrete API-Komposition spart somit Host-Phasenarithmetik, aber weder einen Aufruf noch den sichtbaren Zwischenpuffer. Ein dokumentierter residenter MVM-/Nichtlinearitätspfad wurde im geprüften öffentlichen Vertrag nicht gefunden. Dies schließt weitergehende Möglichkeiten im nicht offenen Hardwaretreiber nicht aus. [36, D7]

Die algebraische Umformung erhält nicht automatisch das Verhalten in endlicher Präzision. Das CPU-Backend rundet jedes MVM-Produkt auf BF16, summiert in FP32 und rundet die Ausgabe erneut auf BF16. Bei getrennten Produkten αx und αy kann die Information eines kleinen Schlüsselabstands bereits vor der Summation verloren gehen. Die zusätzlichen Prüfungen verwenden kein Rauschen, keine weitere Festkommarundung und keinen Toleranzbereich. [36, D7]

Tabelle 13. Affine CPU-Phase im Vergleich zu zwei Kontrollen

Breite	Paare je Pfad	Direkt: Fehler	Host-Phase: Fehler	Affin: Fehler
4 Bit	256	0	0	0
8 Bit	65.536	0	0	40

Quelle D7: summary.json; alle geordneten Paare, einschließlich Gleichständen. SDK 2.3.1, kein Zusatzrauschen.

Beim 8-Bit-Paar (165,166) lauten die drei gerundeten Produkte −1,015625; +1,015625; 1,5703125. Damit entsteht dieselbe Phase wie bei einem Gleichstand. Nach Abzug der einzigen Referenz aus (0,0) ist der Score null. Alle 40 Fehler sind solche falschen Gleichstände; Vorzeichenumkehrungen oder beschädigte echte Gleichstände treten nicht auf. Die kleinste Scoregröße über alle ungleichen 8-Bit-Paare beträgt deshalb null. Der kleinste von null verschiedene Score darf nicht als verfügbare Mindestreserve ausgegeben werden. Referenzkorrektur und Toleranzzone stellen verlorene Information nicht wieder her.

Eine vorher festgelegte Stichprobe verwendet jeweils die ersten 100 Listen aus sechs vorhandenen Datensätzen. Beide Mappings werden mit der affinen Variante und den zwei Kontrollen ausgeführt: insgesamt 3.600 Sortierläufe, davon 1.200 affine. Alle Kontrollsortierungen sind korrekt und stabil. Die Tabelle zeigt die affine Variante; jede Zelle hat 100 Versuche. [D7]

Tabelle 14. Vollständige Sortierkontrolle der affinen Variante

Affine Variante	Bitonic korrekt / stabil	Rang korrekt / stabil
4 Bit, N=16, verschieden	100 / 100	100 / 100
4 Bit, N=16, Duplikate erlaubt	100 / 100	100 / 100
8 Bit, N=16, verschieden	95 / 95	95 / 95
8 Bit, N=16, Duplikate erlaubt	97 / 97	97 / 97
8 Bit, N=256, verschieden	0 / 0	0 / 0
8 Bit, N=256, Duplikate erlaubt	0 / 0	0 / 0

Quelle D7: je Zelle 100 gespeicherte Listen. Alle Ausgaben gültig. Beide Kontrollpfade: jeweils 100 korrekt und stabil in jeder Zelle.

Alle affinen Ausgaben der Stichprobe bleiben gültige Permutationen. Falsche Gleichstände aktivieren jedoch die Originalindexregel für tatsächlich unterschiedliche Schlüssel. Eine gültige Ausgabe ist somit nicht zwangsläufig korrekt sortiert. Die Stichprobenquoten sind weder universelle Fehlerwahrscheinlichkeiten noch Hardwaremesswerte.

7.5 Konsequenz für die Machbarkeitsstudie

Die geprüfte affine Variante wird nicht als korrekter 8-Bit-Komparator übernommen. Das negative Ergebnis ist auf diese Codierung und CPU-Arithmetik begrenzt; andere Codierungen oder eine andere reale Rechenpräzision sind damit nicht ausgeschlossen. Vor weiteren Optimierungen müssen Biasdarstellung, interne Präzision, Eingangsbereiche und ein tatsächlich unterstützter Datenweg geklärt sein. Die bestehende Variante mit Host-Phasenbildung bleibt als korrekt funktionierende Kontrolle erhalten.

Für P Paare betragen die logischen BF16-Puffermengen ohne Kalibrierung oder angenommenes Caching 6P + 4 Byte im direkten Pfad, 12P + 4 Byte bei Host-Phasenbildung und 14P + 6 Byte bei der affinen Variante. Letztere benötigt eine dritte Eingabespalte. Gezählt werden öffentliche

Ein-/Ausgabepuffer und Gewichte, keine gemessenen PCIe-Transfers. Die Verlagerung der Arithmetik bringt im geprüften API-Pfad somit keine belegte Ressourceneinsparung.

Eine nützliche interne Verbindung müsste etwa eine Zwischenkonversion vermeiden und dies bei ausreichender Genauigkeit und geringerem Gesamtaufwand belegen. Auch eine hostfreie Verarbeitung wäre noch keine unregenerierte optische Kaskade. Die öffentliche KAN-Komposition enthält ihrerseits eine explizite Phasenaddition auf der CPU zwischen Treiberaufrufen. Eine höherstufige SDK-Funktion allein weist daher keine solche Kaskade nach. [36]

Die periodische Variante bleibt eine begrenzte Machbarkeitsstudie. Ihre Rolle ist nun durch einen reproduzierbaren Präzisionsbefund und konkrete Herstellerfragen eingegrenzt. Dass eine native nichtlineare Funktion existiert, beweist weder ihre Eignung für dieses Teilproblem noch einen Systemvorteil.

7.6 Herstellerangaben und offene Gerätewerte

Die öffentlich geprüfte Q.ANT-Produktseite nennt PCIe Gen4 x8, 8 GOPS und eine NPU-Leistungsangabe von 150 W sowie einen CPU-Simulationspfad. Diese Angaben beschreiben kein Sortierergebnis und keine garantierte Vergleichsrate. Die Zahl der Verarbeitungskanäle definiert keine quadratische MVM-Dimension. Die tatsächlichen Transferkosten und das kleinste zuverlässig unterscheidbare Signal müssen für den konkreten Kernel bestimmt werden. [37, Zugriff 07.10.2026]

Offen bleiben tcos-Kennlinie, Eingangsbereiche, Auflösung, Drift und Rauschen. Messrunner und Anfrageentwurf sind vorbereitet; physische Messungen und eine Herstellerantwort sind nicht dokumentiert. Kapitel 15 beschreibt die benötigten Daten.

8. Referenzversuch: Eingaben, Quantisierung und Rauschen

8.1 Korpus und Versuchsraum

Die gespeicherten Schlüsselbereiche sind 0 bis 15 und 0 bis 255. Für 4 Bit werden die Größen N = 2, 4, 8, 16 verwendet; für 8 Bit zusätzlich 32, 64, 128 und 256. Jede Kombination aus Breite, N und Eingabefamilie enthält 1.000 Listen. Zwei Familien werden unterschieden: verschiedene Schlüssel und gleichverteiltes Ziehen mit Zurücklegen, bei dem Gleichstände möglich sind. Insgesamt umfasst der Korpus 24.000 Listen in 24 Datensätzen.

Die Familie „Gleichstände erlaubt“ garantiert nicht für jede einzelne kleine Liste einen Gleichstand. Sie bezeichnet den Erzeugungsprozess. Im maximalen Distinct-Fall N = 2^b enthält jede Liste dagegen den gesamten Schlüsselbereich in anderer Reihenfolge.

Eingaben werden vor der Verarbeitung validiert. Ganze numerische Werte wie 3,0 sind zulässig; boolesche Werte, Zeichenketten, Brüche und Werte außerhalb des deklarierten Bereichs werden mit Position abgewiesen. Der gespeicherte Korpus wird unverändert wiederverwendet. Neue Rauschbedingungen sind keine neuen unabhängigen Eingabekorpora.

8.2 Breiten und Normierung

b_key bestimmt den gültigen Schlüsselbereich, b_in die Eingabecodierung und b_out die modellierte Differenzausgabe. Die aktuellen Referenzrunner unterstützen die vereinbarten gleichen Tripel (4,4,4) und (8,8,8). Ungleiche Tripel sind getrennte denkbare Parameter, aber kein ausgewerteter Versuchsraum.

Mit der größten zulässigen Schlüsselzahl M und der Zahl der Eingangsstufen L_in gilt:

$$M = 2^{b_{\text{key}}} - 1, \qquad L_{\text{in}} = 2^{b_{\text{in}}}$$

Die deklarierte Skalierung lautet:

$$u(x) = \frac{x}{M}\left(1 - \frac{1}{L_{\text{in}}}\right), \quad \widehat{x} = \Delta_{\text{in}}\,\text{roundEven}\left(\frac{u(x)}{\Delta_{\text{in}}}\right)$$

Mit Δ_{in} = 1/L_in gilt bei gleichen Schlüssel- und Eingabebreiten exakt $\hat{x} = x/2^b$. Die Skalierung richtet sich nach dem vollständigen deklarierten Bereich, nicht nach Minimum und Maximum einer einzelnen Liste. Dadurch bleiben Signalabstände zwischen Listen vergleichbar.

8.3 Der signorientierte Referenzkomparator

Es gilt $\Delta_{in} = \Delta_{out} = \Delta = 2^{-b}$. Der Komparator bildet die Differenz d = â − b̂, addiert ε aus einer Gaußverteilung mit Standardabweichung σ = ηΔ, rundet zum nächsten geraden Integercode und sättigt auf den b-Bit-Codebereich:

$$c = \text{roundEven}\left(\frac{d+\varepsilon}{\Delta}\right), \quad c_{\text{lim}} = \text{clip}(c, -2^{b-1}, 2^{b-1}-1), \quad \widehat{d} = c_{\text{lim}}\Delta$$

Die sechs η-Werte sind 0; 0,05; 0,10; 0,15; 0,20 und 0,25. Sie sind synthetische Sensitivitätswerte in Einheiten des Differenzrasters. Sie sind weder gemessene Q.ANT-Störungen noch Fehlerwahrscheinlichkeiten oder Prozentwerte einer optischen Leistung.

Ohne Zusatzrauschen ist $d/\Delta = a - b$ ganzzahlig. Runden verändert diesen Code nicht; Sättigung verändert weder sein Vorzeichen noch seinen Nullstatus. Deshalb ist die signbasierte Entscheidung trotz begrenztem Ausgabebereich exakt. Die Aussage gilt nicht für eine Rekonstruktion des Differenzbetrags.

8.4 Ein fehlerhafter Gleichstand erklärt den ersten Ausfall

Für (10,9) bei 8 Bit ist $d = 1/256$. Bei $\varepsilon = +1/1024$ wird $(d+\varepsilon)/\Delta = 1{,}25$ auf 1 gerundet; die Entscheidung bleibt korrekt. Bei $\varepsilon = -3/1024$ wird der Quotient 0,25 auf 0 gerundet. Die Originalindexregel behandelt die ungleichen Schlüssel dann fälschlich als Gleichstand und kann [10,9] stehen lassen.

Im früheren Ausgangsmodell mit $\Delta_{out} = 2\Delta_{in}$ konnten benachbarte Schlüssel bereits ohne Zusatzrauschen auf null runden. Dieses historische Modell bleibt im alten Code zur Herkunft nachvollziehbar, ist aber nicht das aktuelle Referenzexperiment. Die folgenden Ergebnisse verwenden ausdrücklich das gleiche Eingangs- und Ausgangsraster.

8.5 Ablauf und Statistik

Jede Breite-Größe-Familie wird bei sechs Rauschstärken mit zwei Mappings ausgewertet: $24 \times 6 \times 2 = 288$ Konfigurationen mit je 1.000 Listen. Eingaben, Seeds, Modellparameter und Ausgabedateien sind gespeichert. Die Batchgröße beeinflusst den Arbeitsspeicher des Runners; absolute Versuchsschlüssel sorgen dafür, dass sie nicht das Zufallsexperiment verändert.

Die vollständige Korrektheitsquote bezieht sich auf alle Versuche, einschließlich ungültiger Rangfälle. Wilson-Intervalle beschreiben die binäre Erfolgsquote pro Konfiguration. 1.000 von 1.000 Erfolgen entsprechen ungefähr einem 95-%-Intervall von 99,62 % bis 100 %; sie beweisen keine universelle Fehlerfreiheit bei beliebig vielen zukünftigen Eingaben. Die gemeinsame Nutzung von Eingaben und Kontrollstreams darf nicht als unabhängige Wiederholung gezählt werden.

9. Ein gemeinsames Signal- und Rauschmodell

9.1 Warum die früheren Szenarien nicht direkt vergleichbar sind

Im früheren Upstream-Szenario wird die Differenz gestört und anschließend auf Δ gerundet. Im Output-Szenario wird der periodische Ausgang gestört, ohne denselben Rundungsschritt. Außerdem vergrößert die unnormierte Sinuskennlinie den Signalabstand nahe null um ungefähr π/2. Der gleiche Zahlenwert von σ bezeichnet damit nicht die gleiche Entscheidungssituation. Der beobachtete Unterschied „5,9 % gegenüber 100 %“ beschreibt zwei verschiedene Modelle, keine gemessene Überlegenheit einer photonischen Nichtlinearität.

Für benachbarte Schlüssel, d = Δ und σ = 0,25Δ, liegt die direkte Nullschwelle vier Standardabweichungen entfernt. Die vorgelagerte Rundung setzt jedoch bereits bei d + ε < Δ/2 den Code auf null: Diese Grenze liegt nur zwei Standardabweichungen vom Sollwert entfernt. Der ideale unnormierte periodische Abstand beträgt ungefähr 6,28σ; mit der konkret gespeicherten BF16-Kennlinie beträgt der kleinste 8-Bit-Abstand ungefähr 8,00σ. Diese Unterschiede müssen vor einem Robustheitsvergleich beseitigt oder physikalisch begründet werden.

Bei Gaußrauschen beträgt die Wahrscheinlichkeit eines falschen Nullcodes für d = Δ im Referenzmodell ungefähr $\Phi(-2) - \Phi(-6) = 2{,}275\ \%$. Eine echte Vorzeichenumkehr hat dagegen nur ungefähr die Wahrscheinlichkeit $\Phi(-6)$. Bei einem tatsächlichen Gleichstand wird der Nullcode mit Wahrscheinlichkeit $2\Phi(-2) \approx 4{,}55\ \%$ verlassen. Diese Einzelvergleichswerte erklären Fehlertypen; die vollständige Sortierwahrscheinlichkeit folgt daraus wegen abhängiger Abläufe nicht einfach durch Potenzieren.

9.2 Einheitliche Koordinaten und Verarbeitungsschritte

Das neue Kontrollmodell verwendet normierte Differenzeinheiten und eine explizite Reihenfolge. β_{in} und β_{out} sind feste Restoffsets, ε_g ist ein relativer Verstärkungsfehler. Z_in und Z_out sind unabhängige standardnormalverteilte Zufallsgrößen je Vergleich. Q_q rundet mit Schritt q zum nächsten geraden Code; Q_0 ist die Identität.

$$z = \mathrm{clip}_{[-1,1]}\left(Q_{q_{\mathrm{pre}}}(d + \beta_{\mathrm{in}} + \sigma_{\mathrm{in}} Z_{\mathrm{in}})\right)$$

$$y = \mathrm{clip}_{[-1,1]}\left(Q_{q_{\mathrm{out}}}((1+\varepsilon_g)g(z) + \beta_{\mathrm{out}} + \sigma_{\mathrm{out}} Z_{\mathrm{out}})\right)$$

Die beiden idealen Übertragungen sind:

$$g_{\mathrm{direkt}}(d) = d, \qquad g_{\mathrm{periodisch}}(d) = \frac{\sin(\alpha d)}{\alpha}, \qquad \alpha = \frac{\pi}{2}$$

Beide haben am Entscheidungspunkt die Steigung eins. Wird eine reale Ausgangsskala durch α dividiert, müssen ihre Störanteile ebenfalls dividiert werden. Die Normierung erzeugt keinen kostenlosen Präzisionsgewinn. Am Rand ist der periodische Verlauf flacher; die Gleichheit der Steigung nahe null macht nicht die gesamte Kennlinie identisch.

Das Kontrollmodell wird in Float64 ausgeführt. Es ist weder das BF16-SDK noch ein digitaler Zwilling der Q.ANT-Hardware. Das Clipping auf [−1,1] erzwingt hier einen betrachteten monotonen Bereich. Ein symmetrischer Quantisierer mit Schritt Δ über diesem gesamten Bereich besitzt mehr als 2^b Codes; er wird deshalb nicht als b-Bit-ADC bezeichnet.

Tabelle 15. Parameter und ihr Evidenzstatus

Größe	Gespeicherte Kontrolle	Begründung / Grenze
Schlüsselraster Δ	2^{-4} oder 2^{-8}	Normierung des Integerkorpus
σ_{in} / Δ; σ_{out} / Δ	0 oder 0,25	Angenommene Sensitivität, keine Messung
q_pre	0 oder Δ	Keine oder gemeinsame Zwischenrundung
q_out	0; 0,5Δ; Δ; 2Δ	Getrennte Auflösungsannahme in Paarfällen
β_{out}	−0,25Δ; 0; +0,25Δ	Fester Restoffset, keine Drifttrajektorie
β_{in}; ε_g	0; 0	Unterstützte Effekte hier deaktiviert
Kennlinie und Bereich	d bzw. sin(πd/2)/(π/2); [−1,1]	Ideale Float64-Funktionen, kein tcos-Fit
Rauschverteilung	Unabhängiges Gaußrauschen	Schwere Tails und Korrelationen noch offen

Quelle: results/common_noise_20261007/metadata.json und docs/COMMON_NOISE_MODEL.md. Nicht jede Kombination wird als vollständige Sortierung getestet.

Die gespeicherten Sortierkontrollen nutzen ausgewählte Kombinationen der Parameter. Die umfassenderen Paarprüfungen variieren zusätzlich Restoffset und Ausgaberaster. Eingangsversatz und Verstärkungsfehler sind in diesen gespeicherten Kontrollen null. Feste Offsets sind keine zeitabhängige Drift; eine Drifttrajektorie wurde bislang nicht gemessen oder simuliert.

9.3 Wie belegte physikalische Werte später eingehen

López-March et al. analysieren Rauschen in APC und motivieren die Trennung physikalischer Quellen, ihrer Übertragung und ihrer Korrelationen. In einem konkreten Beispiel werden unter anderem 1.550 nm, eine Laserlinienbreite von 400 kHz und ein RIN-Wert von −145 dB/Hz angegeben. Diese Werte gehören zum dort untersuchten Aufbau. Sie liefern weder die Empfängerbandbreite noch eine Q.ANT-Komparatorvarianz und werden hier nicht als Q.ANT-Simulationsparameter eingesetzt. [21, S. 8 der bereitgestellten Fassung]

Für einen idealisierten Empfänger kann eine Stromrauschvarianz bei flachen Spektraldichten beispielsweise so aufgebaut werden:

$$\sigma_I^2 = B_{\mathrm{eq}}\left(S_{\mathrm{th}} + 2q_e I + 10^{\mathrm{RIN_{dB}}/10} I^2\right)$$

I ist dabei der positive Photostrom eines Detektorzweigs, nicht einfach eine vorzeichenbehaftete Differenz. B_eq ist die äquivalente Rauschbandbreite; q_e die Elementarladung. Für differenzielle Auslese, mehrere Kanäle und korrelierte Quellen ist die vollständige Kovarianz über die kalibrierte Übertragung zu propagieren. Im linearisierten Fall lautet dies $J\Sigma J^{\mathrm{T}}$. Die Laserlinienbreite darf nicht als B_eq eingesetzt werden.

Für die vereinfachte skalare Kennlinie ergibt sich lokal bei unabhängigen Eingangs- und Ausgangsstörungen:

$$\mathrm{Var}(y \mid d) \approx (1+\varepsilon_g)^2 [g'(d)]^2 \sigma_{\mathrm{in}}^2 + \sigma_{\mathrm{out}}^2$$

Quantisierung, Clipping und Nichtlinearität können diese linearisierte Näherung begrenzen. Bei Referenzsubtraktion muss zusätzlich Var(y − y_ref) = Var(y) + Var(y_ref) − 2 Cov(y,y_ref) berücksichtigt werden. Ein einmal gemessener Referenzfehler ist über viele Vergleiche gemeinsam

wirksam. Ihn unabhängig pro Vergleich neu zu ziehen würde eine andere Hardwareannahme erzeugen.

9.4 Kontrollen vor großen Sortierläufen

Die vorhandene Kontrollserie prüft 280 Paarfälle mit je 100.000 Stichproben, insgesamt 28 Millionen Entscheidungen. Für 182 geeignete Fälle werden analytische Wahrscheinlichkeiten mit Monte-Carlo-Ergebnissen abgeglichen. Die Toleranz berücksichtigt sechs Standardfehler und einen kleinen endlichen Stichprobenzuschlag; sie ist ein Implementierungstest, keine Aussage über Q.ANT.

Bei gemeinsamer Zwischenquantisierung sind 1,4 Millionen paarweise verglichene Entscheidungen direkt und ideal-periodisch identisch. Eine zusätzliche Sortierkontrolle umfasst 144 Konfigurationen und 14.400 Sortierungen: sechs Datensätze, sechs Einstellungen, zwei Funktionen und zwei Mappings, jeweils die ersten 100 gespeicherten Listen. Die 24 rauschfreien Konfigurationen sind korrekt und stabil. Bei 1.200 passend gepaarten Sortierausgaben mit vorgelagerter Quantisierung sind die Ausgaben ebenfalls identisch. Diese Kontrollen begründen, warum Modellierung und Signalabstand vor einem physikalischen Robustheitsvergleich vereinheitlicht werden müssen.

10. Qualitätsmetriken: vollständige Ordnung und brauchbare Rangfolge

10.1 Vier unterschiedliche Fragen

Eine Ausgabe ist **gültig**, wenn sie jeden ursprünglichen Datensatz genau einmal enthält. Sie ist **korrekt sortiert**, wenn ihre Schlüssel monoton sind. Sie ist **stabil**, wenn gleiche Schlüssel ihre ursprüngliche Reihenfolge behalten. Eine teilweise fehlerhafte Ausgabe kann dennoch eine sehr gute näherungsweise Rangfolge besitzen. Diese Eigenschaften werden getrennt ausgewertet; eine ungültige Ausgabe wird nicht nachträglich zu einer Permutation repariert.

Die vollständige Korrektheit bleibt für die exakte Sortieraufgabe notwendig. Sie beantwortet jedoch nicht, ob ein einzelnes Nachbarpaar oder große Teile der Liste falsch angeordnet sind. Kendall-τ, Inversionen nach Schlüsselabstand und Recall@k ergänzen deshalb die binäre Erfolgsquote.

10.2 Kendall-τ-b bei gleichen Schlüsseln

Für eine gültige Ausgabe werden ihre Positionen mit den zugehörigen Originalschlüsseln verglichen. Es gibt C = N(N−1)/2 Positionspaare. M davon haben ungleiche Schlüssel; I dieser M Paare sind invertiert. Weil die Positionen keine Gleichstände haben, die Schlüssel aber schon, gilt:

$$\tau_b = \frac{M - 2I}{\sqrt{CM}}, \qquad M > 0$$

Sind alle Schlüssel verschieden, ist M = C und eine perfekte Ordnung hat $\tau_b = 1$. Bei Duplikaten beträgt die erreichbare Obergrenze dagegen √(M/C), auch wenn kein ungleiches Paar invertiert ist. Das ist eine Folge der Bindungskorrektur, kein Sortierfehler. Der Bericht zeigt deshalb zusätzlich die Obergrenze und, wo hilfreich, die ausdrücklich anders benannte Größe $\tau_{key} = 1 - 2I/M$. Sie normiert nur die ungleichen Schlüsselpaare und ist nicht als Standard-Kendall-τ-b auszugeben. [39]

Für ausschließlich gleiche Schlüssel ist M = 0 und die Metrik undefiniert. Ungültige Rangausgaben haben ebenfalls kein τ in dieser Auswertung. Mittelwerte über gültige Ausgaben werden immer zusammen mit ihrer Anzahl berichtet; sie dürfen einen hohen Anteil ungültiger Ausgaben nicht verbergen.

10.3 Fehler nach Schlüsselabstand

Für jedes ungeordnete Paar verschiedener Originalschlüssel wird der ganzzahlige Abstand g = |a − b| gebildet. Die Auswertung zählt pro g die invertierten und die insgesamt vergleichbaren Paare in gültigen Ausgaben. Ihre Division ist eine bedingte Inversionsquote. Sie zeigt, ob die Ausgabe hauptsächlich nahe beieinanderliegende Schlüssel verwechselt.

Diese Metrik rekonstruiert keine internen Komparatorfehler. Ein gestörter Vergleich kann später korrigiert werden oder andere Datenbewegungen auslösen. Aus der endgültigen Permutation lassen sich nur deren Ordnung und Inversionen, nicht sämtliche früheren Entscheidungen ablesen. Ungültige Rangfälle gehen in die separate Gültigkeitsquote ein und haben hier keinen Paar-Nenner.

10.4 Recall@k und eine faire Behandlung der Auswahlgrenze

Die Ausgaben sind aufsteigend sortiert; für die Top-k-Auswahl werden deshalb die letzten k Positionen verwendet. Verglichen wird mit den größten k Elementen der stabilen digitalen Referenz. Die gespeicherten k-Werte sind 1, 5, 10 und 32, soweit k ≤ N, sowie N als Vollmengen-Kontrolle.

Die ID-basierte Variante zählt die Schnittmenge der ausgewählten Originalindizes mit der stabilen Referenz und dividiert durch k. Bei einem Gleichstand an der Auswahlgrenze kann eine andere Auswahl gleichwertiger Datensätze diese Kennzahl senken. Die bindungstolerante Variante schreibt deshalb alle ausgewählten Schlüssel oberhalb der Grenze gut und erlaubt für den Grenzschlüssel genau die noch benötigte Anzahl. Sie überschreitet nie eins.

Es werden sowohl der Mittelwert über gültige Ausgaben als auch ein Mittelwert über alle Versuche angegeben, in dem ungültige Ausgaben null beitragen. Recall@N ist für jede gültige Permutation eins und prüft lediglich die Mengenerhaltung. Es ersetzt keine Ordnungsmetrik.

10.5 Bezug zu PRISM und Grenzen des Vergleichs

PRISM untersucht die Auswahl relevanter KV-Cache-Blöcke für lange LLM-Kontexte mittels photonischer Ähnlichkeitsbewertung. Eine ausreichend gute Auswahl kann dort genügen, obwohl keine vollständig exakte Ordnung erzeugt wird. Das motiviert die Top-k-Perspektive. Die vorliegende Sortierauswertung nutzt jedoch andere Daten und ein anderes Anwendungskriterium; ihr Recall ist kein PRISM-Benchmark und sagt nichts unmittelbar über LLM-Qualität aus. Ein späterer Anwendungsvergleich benötigt dieselben Scores, Auswahlgrößen und Qualitätsziele. [23]

10.6 Verfügbare Ausgaben und Auswertungslücken

Für die nachträgliche Qualitätsauswertung waren 1.147 von 1.152 vorgesehenen Konfigurationen auswertbar: 288 Referenzkonfigurationen, 576 periodische Upstream-/Output-Konfigurationen und 283 von 288 Deadband-Konfigurationen. Das sind 1.147.000 ausgewertete Sortierausgaben. Die 288 periodischen Upstream-Konfigurationen reproduzieren die Referenz und sind keine zusätzliche unabhängige Evidenz.

Das damalige Deadband-Archiv besitzt kein vollständiges ZIP-Zentralverzeichnis. CRC-geprüfte vollständige Einträge konnten gelesen werden, ohne Daten zu verändern. Für fünf Konfigurationen mit 8 Bit, N = 256 und zugelassenen Duplikaten fehlen jedoch die benötigten Originalindex-Ausgaben: Rangsortierung bei $\eta = 0{,}15$ sowie beide Mappings bei $\eta = 0{,}20$ und 0,25. Dafür werden keine τ-, Abstands- oder Recall-Werte erfunden. Die gespeicherte coverage.csv benennt diese Lücken.

Die abgeleiteten Qualitäts-CSV-Dateien und ihre Provenienz sind im Repository enthalten. Die älteren vollständigen periodischen Roharchive selbst sind im aktuellen Git-Stand nicht enthalten. Eine erneute Auswertung genau dieser historischen Ausgaben benötigt daher die ursprünglichen Archive; neue Läufe sind ein neues, gesondert zu kennzeichnendes Experiment. Die Referenz- und gemeinsamen Rauschkontrollen enthalten dagegen ihre Ausgaben im Repository.

11. Ergebnisse aus den gespeicherten Versuchen

11.1 Umfang und Aussageebenen

Tabelle 16. Vorhandene Versuchsserien

Serie	Umfang	Prüfzweck / Status
Referenz, sechs η	288 Konfigurationen; 288.000 Sortierungen	Gespeicherte vollständige Ausgaben
Lineares CPU-SDK	48.000 rauschfreie Sortierungen	Softwareintegration, kein Timing
Periodische Paare	256 + 65.536 geordnete Paare	Vollständige endliche Domänenprüfung
Affine CPU-Phase	3 × (256 + 65.536) Paare; 3.600 Sortierungen	Begrenzte Machbarkeitskontrolle mit zwei Kontrollpfaden
Periodische Szenarien	864 Konfigurationen; nominell 864.000 Sortierungen	Historische Serie; Roharchive fehlen im Git-Stand
Zusätzliche Rankingmetriken	1.147 / 1.152 Konfigurationen; 1.147.000 Ausgaben	Referenz + historische Szenarien; 5 Lücken
Gemeinsames Rauschmodell	28 Mio. Paarentscheidungen; 14.400 Sortierungen	Synthetische Kontrollen, eigenes Protokoll
CPU-Baseline	192 Konfigurationen; 1.728 Zeitwiederholungen	24.000 Eingaben; Batch- und Typvarianten

Die Tabelle fasst unterschiedliche Prüfzwecke zusammen. Paarprüfungen, wiederverwendete Kontrollstreams und wiederholt zeitlich vermessene CPU-Eingaben werden nicht zu einer vermeintlich unabhängigen Gesamtstichprobe addiert. Die ausgewählten Tabellen dieses Kapitels werden beim Berichtsbau direkt aus den gespeicherten Ergebnisdateien gelesen. Die vollständigen CSV-Dateien bleiben im Repository verfügbar.

11.2 Referenz: vollständige Korrektheit sinkt mit vielen nahen Vergleichen

Im Referenzversuch sind bei $\eta \leq 0{,}10$ in jeder der geprüften Konfigurationen alle 1.000 Sortierungen korrekt und stabil. Bei stärkeren Störungen treten zunächst einzelne nahe Fehlordnungen auf. Für 8 Bit und $N = 256$ enthält die Familie verschiedener Schlüssel sämtliche Werte des Wertebereichs; viele Schlüssel liegen direkt nebeneinander. Eine einzige Fehlordnung genügt, um die gesamte Liste als nicht korrekt zu zählen.

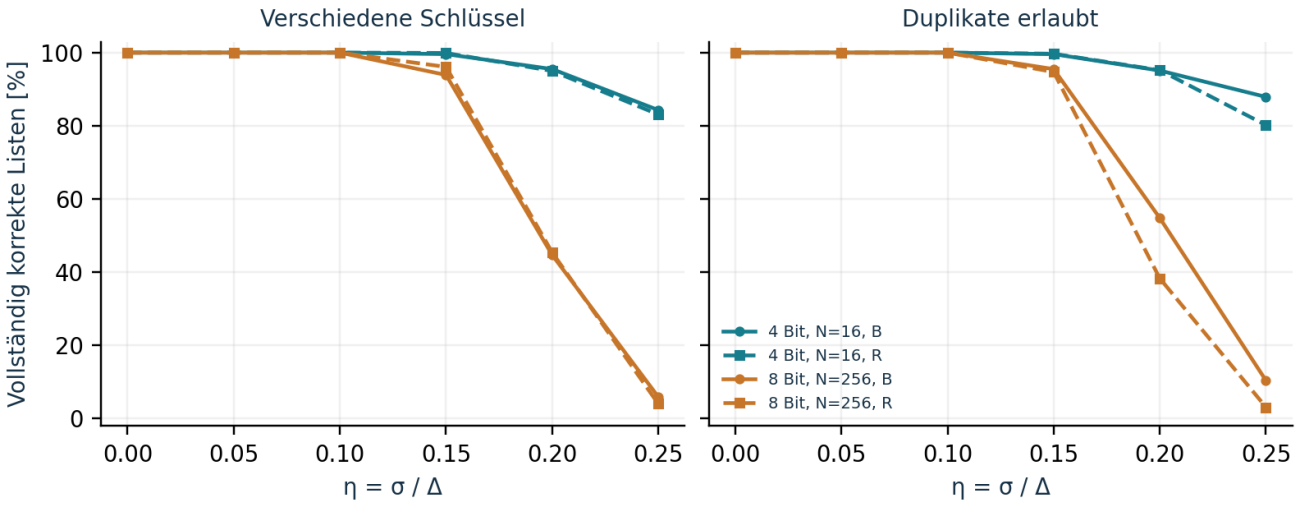


Abbildung 1. Referenzmodell bei maximaler getesteter Listengröße. B: Bitonic, R: Rangsortierung; je Punkt 1.000 Listen. Quelle: results/reference/accuracy.csv.

Tabelle 17. Vollständige Korrektheit bei maximaler getesteter Listengröße

Breite / N	Eingaben	η	Bitonic korrekt	Rang korrekt
4 Bit / 16	verschieden	0,15	99,6 %	99,9 %
4 Bit / 16	verschieden	0,20	95,5 %	95,0 %
4 Bit / 16	verschieden	0,25	84,3 %	83,1 %
4 Bit / 16	Duplikate erlaubt	0,15	99,6 %	99,7 %
4 Bit / 16	Duplikate erlaubt	0,20	95,1 %	95,2 %
4 Bit / 16	Duplikate erlaubt	0,25	87,9 %	80,2 %
8 Bit / 256	verschieden	0,15	93,9 %	96,1 %
8 Bit / 256	verschieden	0,20	44,7 %	45,4 %
8 Bit / 256	verschieden	0,25	5,9 %	4,0 %
8 Bit / 256	Duplikate erlaubt	0,15	95,5 %	94,7 %
8 Bit / 256	Duplikate erlaubt	0,20	54,8 %	38,3 %
8 Bit / 256	Duplikate erlaubt	0,25	10,4 %	3,1 %

Quelle: results/reference/accuracy.csv; je Zelle 1.000 Versuche. Ungültige Ausgaben zählen als Fehler.

Bei zugelassenen Duplikaten ist Stabilität besonders empfindlich: Im 8-Bit-Fall mit N = 256 und η = 0,25 sind bei Bitonic 10,4 % der Listen nach Schlüsseln korrekt, aber nur 0,5 % stabil; bei Rangsortierung sind es 3,1 % beziehungsweise 0,6 %. Ein Rauschen, das einen wahren Gleichstand aufbricht, kann die Schlüsselordnung erhalten und trotzdem die Originalreihenfolge zerstören.

11.3 Gute Rangfolge trotz geringer vollständiger Korrektheit

Tabelle 18. 8 Bit, N = 256, verschiedene Schlüssel, η = 0,25

Kennzahl	Bitonic	Rangsortierung
Gültige Ausgaben / 1.000	1000	984
Vollständig korrekt	5,9 %	4,0 %
Mittleres τ_b, gültige Ausgaben	0,999826	0,999823
Mittlere Inversionen, gültige Ausgaben	2,847	2,889
Recall@10, gültige Ausgaben	99,880 %	99,878 %
Recall@10, ungültige Ausgaben = 0	99,880 %	98,280 %

Quellen: results/ranking_quality_20261007/summary.csv und recall.csv. Bei verschiedenen Schlüsseln stimmen die beiden Recall-Varianten überein.

Die Tabelle bezieht sich einheitlich auf 8 Bit, N = 256, verschiedene Schlüssel und η = 0,25 im Referenzmodell. Bei Bitonic liegen im Mittel nur 2,847 Inversionen unter 32.640 möglichen Paaren vor. Von den insgesamt 2.847 beobachteten Inversionen entfallen 2.834 auf Abstand eins und 13 auf Abstand zwei; bei größeren Abständen wurden keine beobachtet. Rund 99,54 % der Inversionen betreffen somit direkte Nachbarn. Das erklärt die Kombination aus 5,9 % vollständig korrekten Arrays und fast perfektem τ beziehungsweise Recall.

Bei Rangsortierung sind 984 von 1.000 Ausgaben gültige Permutationen. Ihr gutes bedingtes Recall@10 von ungefähr 99,878 % muss deshalb zusammen mit dem Wert von 98,28 % über alle Versuche gelesen werden. Das Auslassen der 16 ungültigen Fälle würde die praktische Auswahlqualität günstiger erscheinen lassen, als sie unter dem definierten Ausgabevertrag ist.

11.4 Periodischer Vergleich und Gleichstände

Die rauschfreien Paarprüfungen bestätigen nach Referenzkorrektur die Entscheidung für alle 256 geordneten 4-Bit-Paare und 65.536 geordneten 8-Bit-Paare. Die historischen vollständigen Sortierauswertungen bestätigen ebenfalls rauschfreie Korrektheit und Stabilität für die geprüften Konfigurationen. Daraus folgt keine neue Hardwareleistung: Die Funktion läuft im CPU-Backend, und ihre Phase wird auf dem Host vorbereitet.

Besonders aufschlussreich ist das frühere Output-Szenario mit Duplikaten. Bei 8 Bit, N = 256 und η = 0,25 sind die Bitonic-Ausgaben nach Schlüsseln zu 100 % korrekt, aber keine ist stabil. Recall@10 nach stabilen IDs beträgt 97,76 %, bindungstolerant dagegen 100 %. Das mittlere τ_b von ungefähr 0,998046 liegt an seiner bindungsbedingten Obergrenze; τ_{key} ist eins. Die Rangsortierung erzeugt in dieser Konfiguration keine gültigen Ausgaben. Gleichstände sind somit keine unwichtige Randbedingung des Vergleichsbausteins.

Ein tolerierter Nullbereich kann gestörte Gleichstände auffangen. Er kann aber zugleich benachbarte unterschiedliche Schlüssel als gleich behandeln. Bei symmetrischer Toleranz τ und minimalem ungleichem Signalabstand m schützt ein begrenzter Gesamtfehler kleiner als $\min(\tau, m - \tau)$ die Dreiwegeentscheidung. $\tau = m/2$ balanciert diese zwei Abstände. Unbeschränktes Gaußrauschen liefert nur Fehlerwahrscheinlichkeiten, keine deterministische Garantie. Der CPU-Wert für m ist nicht auf eine ungemessene tcos-Kennlinie übertragbar.

11.5 Gemeinsames Modell: was die Kontrollen tatsächlich zeigen

Tabelle 19. Gemeinsames Modell: ausgewählte vollständige Sortierungen

Einstellung	Mapping	Gültig / 100	Korrekt / 100	τ_b gültig	Recall@10 alle
Gemeinsame Zwischenrundung	Bitonic	100	8	0,999814	99,8 %
Gemeinsame Zwischenrundung	Rang	100	4	0,999819	99,8 %
Ein- und Ausgangsrauschen	Bitonic	100	0	0,999371	99,3 %
Ein- und Ausgangsrauschen	Rang	72	0	0,999384	71,8 %
Grobes Ausgaberaster 2Δ	Bitonic	100	0	0,996376	97,3 %
Grobes Ausgaberaster 2Δ	Rang	0	0	–	0,0 %

Quelle: results/common_noise_20261007/sort_quality.csv. 8 Bit, N = 256, verschiedene Schlüssel. Direkte und ideal-periodische Funktion stimmen in diesen ausgewählten Kennzahlen überein. „Alle“ zählt ungültige Ausgaben mit null.

Die ausgewählten Fälle verwenden 8 Bit, N = 256, verschiedene Schlüssel und je 100 Listen. Direkte und ideal-periodische Übertragung liefern in diesen dargestellten Fällen dieselben ausgewerteten Kennzahlen. In der Einstellung mit ausschließlich vorgelagertem Rauschen und gemeinsamer Quantisierung ist dies durch die Entscheidungskontrollen abgesichert. Für andere Einstellungen ist Gleichheit beobachteter Kennzahlen kein allgemeiner Identitätsbeweis.

Zusätzliches Ausgangsrauschen senkt die vollständige Korrektheit; Rangkonflikte können die Gültigkeit stark reduzieren. Ein grobes Ausgaberaster von 2Δ erzeugt Fehlgleichstände bereits durch die Auflösung. Die Ergebnisse belegen daher die Bedeutung der gesamten Signalverarbeitung. Sie belegen keinen belastbaren Unterschied zwischen realen analogen Plattformen.

11.6 Digitale CPU-Baseline

Die digitale Baseline sortiert dieselben 24.000 gespeicherten Eingaben mit NumPy argsort(kind="stable") und sammelt die zugehörigen Schlüssel ein. Sie liefert sowohl sortierte Schlüssel als auch Originalindizes und erfüllt damit denselben stabilen Ausgabevertrag. Alle Eingaben wurden außerhalb des Messbereichs auf Korrektheit und Stabilität geprüft. [38]

Gemessen wurden 192 Konfigurationen: 24 Datensätze × zwei Speicherformate × vier Batchgrößen. Jede besitzt neun Zeitwiederholungen nach zwei Aufwärmläufen und einer Kalibrierung auf ungefähr 15 ms Messdauer. Die Reihenfolge der Konfigurationen wurde reproduzierbar gemischt. Eingabedatei-Lesen, Typumwandlung und Validierung sind ausgeschlossen; Python-Aufruf, Ausgabearray-Allokation, Sortierung und Einsammeln sind enthalten. Der Versuch beschreibt einen aufgewärmten Arbeitsablauf, keinen Kaltstart aus einer Datei.

Die Umgebung meldet einen AMD EPYC 9V74, Python 3.12.14 und NumPy 2.3.5 auf Linux x86-64. Neun logische CPUs sind sichtbar, die Cgroup-Quote beträgt acht CPU-Äquivalente. Daraus folgt nicht, dass die Sortierfunktion acht Threads nutzt. Es wurde keine Prozessorbindung gesetzt.

Tabelle 20. Digitale Baseline: 8 Bit, N = 256, verschiedene Schlüssel

Typ	Listen je Batch	Batchzeit [µs]	Amortisiert [µs/Liste]	Rel. IQR
int64	1	11,90	11,90	5,4 %
int64	1.000	8.851,66	8,85	12,2 %
uint8	1	5,26	5,26	10,6 %
uint8	1.000	1.407,79	1,41	3,7 %

Quelle: results/cpu_baseline_20261007/summary.csv. Median aus neun Wiederholungen; CPU- und Softwareumgebung siehe Text. Kein NPU-Vergleich.

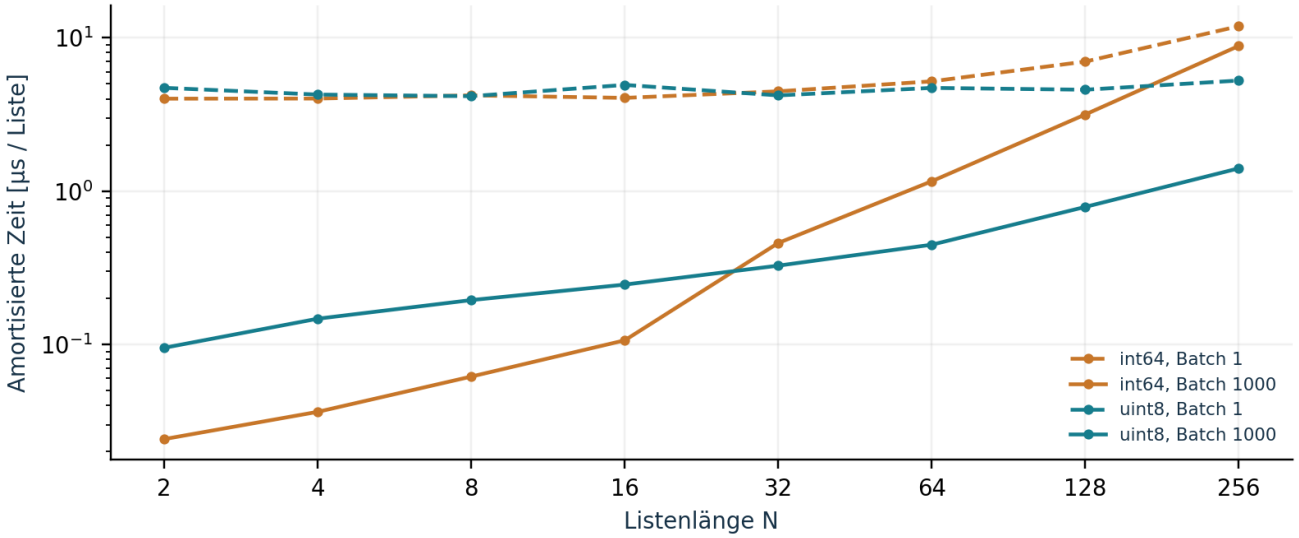


Abbildung 2. CPU-Baseline für 8-Bit-Eingaben mit verschiedenen Schlüsseln. Amortisierte Batchzeiten, keine Einzelanfragelatenzen. Quelle: results/cpu_baseline_20261007/summary.csv.

Angegeben ist der Median der neun gemessenen mittleren Batchzeiten. Die amortisierte Zeit pro Liste ist Batchzeit geteilt durch Batchgröße, keine einzeln gemessene Antwortzeit. Der Interquartilsabstand beschreibt die Streuung dieser Wiederholungen; er ist weder Konfidenzintervall noch Verteilung einzelner Anfragelatenzen. Vier von 192 Konfigurationen haben einen relativen Interquartilsabstand über 20 %; sie wurden nicht entfernt.

Die Unterschiede zwischen uint8 und int64 zeigen, dass Speicherformat und verwendeter Sortierpfad relevant sind. Diese Baseline ist reproduzierbar und erfüllt den Vertrag, beansprucht aber nicht, die schnellste denkbare CPU-Implementierung zu sein. Ohne einen gemessenen vollständigen NPU-Lauf ist daraus kein photonischer Beschleunigungsfaktor ableitbar. Eine GPU-Baseline und Energieaufzeichnung fehlen weiterhin.

12. Aufrufe, Datenbewegung und mögliche Systemkosten

12.1 Logische Zählung statt erfundener Gerätekapazität

Die folgenden Zählungen beschreiben das konkret vorgesehene SDK-Mapping. Sie zählen Nutzdaten und Aufrufe, nicht tatsächlich gemessene PCIe-Transaktionen. Für eine Kapazität c linearer Paaroperationen je Aufruf gilt bei stufenweiser Bitonic-Ausführung:

$$q_B = K\left\lceil \frac{N/2}{c} \right\rceil, \qquad q_R = \left\lceil \frac{C_R}{c} \right\rceil$$

c ist eine Kapazitätsannahme, kein öffentlich bestätigtes Q.ANT-Limit. BF16-Eingaben benötigen logisch vier Bytes je Paar und die Differenzausgabe zwei Bytes. Bei erneut übergebenen zwei Gewichten entstehen zusätzlich vier Bytes pro linearem Aufruf. Pufferverwaltung, Treiberprotokoll und intern wiederverwendete Daten sind darin nicht enthalten.

Tabelle 21. Direktes lineares Mapping bei vollständig passenden Chargen

N	Mapping	Vergleiche	Lineare Aufrufe	Logische Bytes
16	Bitonic	80	10	520
16	Rang	120	1	724
64	Bitonic	672	21	4.116
64	Rang	2.016	1	12.100
256	Bitonic	4.608	36	27.792
256	Rang	32.640	1	195.844

Eigene Zählung: 4 Eingabebytes + 2 Ausgabebytes je BF16-Paar; 4 Gewichtsbytes je Aufruf. Keine gemessenen Treiber- oder PCIe-Bytes.

Für N = 256 und eine genügend große einzelne Rangcharge stehen 36 lineare Bitonic-Aufrufe einem Rangaufruf gegenüber. Rangsortierung überträgt dafür rund siebenmal so viele logische Nutzbytes. Bei c = 128 benötigt bereits die lineare Rangvariante 255 Aufrufe, Bitonic weiterhin 36; mit je einem zusätzlichen periodischen Aufruf pro Teilcharge sind es 510 gegenüber 72. Der vermeintliche Ein-Aufruf-Vorteil hängt also wesentlich von der realen Kapazität ab.

12.2 Zusätzlicher periodischer Hostpfad

Der vorhandene periodische Ablauf liest die lineare Differenz zurück, berechnet die Phase elektronisch, übergibt Phase und Amplitude an einen weiteren Aufruf und liest dessen Ausgang zurück. Dies erzeugt zusätzliche Arrays, Aufrufe und Datenverkehr. Bei vollständig passenden Chargen und N = 256 ergeben sich 72 Aufrufe und 55.440 logische Bytes für Bitonic beziehungsweise zwei Aufrufe und 391.684 Bytes für Rangsortierung. Eine einmalige Referenzauswertung kommt gesondert hinzu; ein Referenzpaar umfasst sechs logische Bytes in diesem vereinfachten Nutzdatenmodell.

Da sign(d) auf dem Host bereits verfügbar wäre, muss eine alternative Kaskade eine konkrete Aufgabe übernehmen: etwa die Entscheidung vor einer elektronischen Zwischenübertragung bilden oder innerhalb einer bestätigten Pipeline mehrere Schritte zusammenfassen. „Hostfrei“ ist dabei nicht

gleichbedeutend mit „vollständig optisch“. Ein Nutzen ist erst mit dem tatsächlichen Ausführungspfad und einem gleichwertigen Qualitätsziel bewertbar.

12.3 Eine bedingte Laufzeitgleichung

Für einen vollständig seriellen Ablauf ohne Überlappung kann die physische Zeit einer Architektur A modellhaft zerlegt werden:

$$t_A = q_A \ell + C_A t_{\text{arith}} + \frac{U_A}{B_{\text{up}}} + \frac{V_A}{B_{\text{down}}} + h_A$$

ℓ ist der fixe Aufrufaufwand; U und V sind übertragene Nutzbytes; B_up und B_down effektive Bandbreiten; h_A ist die Hostarbeit einschließlich Packen, Entscheidungen, Rängen und Datenbewegung. Diese Hilfsgrößen beschreiben eine spätere physische Messung und erweitern **nicht** den Komplexitätsvektor C. Gleichzeitige Transfers, Caching und echte Parallelverarbeitung erfordern ein angepasstes Ablaufmodell.

Unter denselben linearen Nutzdatenannahmen wäre Rangsortierung nur dann schneller als Bitonic, wenn eingesparter fixer Aufrufaufwand die zusätzlichen Vergleiche, Nutzbytes und Hostkosten überwiegt. Aus weniger API-Aufrufen allein folgt keine geringere Gesamtlaufzeit. Ebenso ist die Datenblatt-Spitzenbandbreite eines PCIe-Links keine gemessene effektive Anwendungsbandbreite.

12.4 Energie braucht eine klare Messgrenze

Die Literatur zeigt, warum optische Rechenoperationen und vollständige Systeme unterschiedlich bewertet werden müssen. Laser, Verluste, Modulation, ADC/DAC, Speicher, Host und Kühlung können den Energiebedarf bestimmen. Eine bauteilbezogene Zahl oder hohe interne Parallelität lässt sich nicht unmittelbar in Joule je vollständig sortierter Liste umrechnen. [1, 22, 33]

Eine spätere Energiebaseline muss dieselben Ein- und Ausgaben und dieselbe Qualitätsanforderung erfüllen. Zu protokollieren sind Messgrenze, aktive und gegebenenfalls Leerlaufleistung, Zeitintegration, Batchgröße und Unsicherheit. Ob Leerlaufenergie abgezogen wird, muss ausdrücklich festgelegt und zusätzlich die Gesamtenergie angegeben werden. Die aktuellen Herstellerwerte von 8 GOPS und 150 W sind Spezifikationsangaben, keine Messung dieses Sortierworkloads. [37]

13. Vom Konzept zum ausführbaren Code

13.1 Die aktuelle Ausführungskette

Die aktuelle Implementierung liegt im Paket qant_sorting. Wiederverwendbare Modelle, Sortierabläufe und Qualitätsmetriken stehen auf dessen erster Ebene; ausführbare Versuche liegen unter experiments. Der zentrale Einstieg lautet python -m qant_sorting COMMAND. COMMAND --help zeigt die jeweiligen Optionen. Die folgenden Dateinamen beziehen sich auf diese Paketstruktur; ältere Metadaten behalten ihre ursprünglichen Pfade und Quellhashes.

Die Datensätze werden einmal erzeugt und danach identisch wiederverwendet. Das Referenzmodell stellt einen kontrollierten Komparator bereit; die Sortierlogik hält Originaldatensätze und Indizes getrennt vom gestörten Signal. Ein SDK-Adapter realisiert denselben linearen Paarbaustein im offiziellen CPU-Backend. Die periodische Variante ergänzt Phase, Referenzkorrektur und optional einen Nullbereich. Qualitätsauswertung und CPU-Benchmark lesen die gespeicherten Eingaben beziehungsweise Ausgaben, ohne sie stillschweigend zu reparieren.

Die separate affine CPU-Kontrolle in check_affine_periodic.py ergänzt zwei Vergleichspfade um die Phasenbildung in der MVM. Sie speichert auch fehlgeschlagene Paarentscheidungen und Sortierungen unverändert; sie ersetzt keinen vorhandenen Komparator.

Das gemeinsame Rauschmodell ist ein eigener kontrollierter Komparator. Es ersetzt nicht automatisch sämtliche älteren Experimente. Seine Dateien tragen eigene Einstellungen, Provenienz und Kontrollnachweise, damit synthetische Float64-Resultate nicht mit SDK- oder Hardwareausgaben verwechselt werden.

Tabelle 22. Codeübersicht nach Aufgabe

Dateien	Funktion und Grenze
input_validation.py	Schlüsseltypen, Wertebereiche und Eingabeformen prüfen
comparison.py; sorting_schedules.py; metrics.py	Gleichschrittiger Komparator, aktive Bitonic-Stufen, Ränge, Gültigkeit, Ordnung und Stabilität
run_noise_sweep.py; verify_results.py	Referenzläufe, Seeds, gespeicherte Ausgaben und exakter Reproduktionsvergleich
sdk_mapping.py; run_sdk_control.py; install_sdk.py	Gepinnte CPU-SDK-Installation, BF16-Differenz und rauschfreie Integration
periodic_comparison.py; run_periodic_evaluation.py	Phasencodierung, Referenzkorrektur, historische Szenarien und Nullbereich
check_periodic_pairs.py; check_sdk_nonlinearity.py	Endliche Paarprüfung und getrennte ReLU-/Min-Max-Kontrolle
check_affine_periodic.py	Affine CPU-Phasenbildung, erschöpfende Paare und begrenzte vollständige Sortierkontrolle
measure_periodic_curve.py	Wiederholte API-Ausgaben, Codes und Zeiten; explizite Backendprüfung
ranking_quality.py; evaluate_ranking_quality.py; saved_output_reader.py	τ, Inversionen, Recall und CRC-geprüfte eingeschränkte Archivlesung
common_noise_model.py; run_common_noise_controls.py	Gemeinsame Einheiten, explizite Störstellen und analytische / Monte-Carlo-Kontrollen

Dateien	Funktion und Grenze
run_cpu_baseline.py	Stabile digitale Sortierung, Kalibrierung, Batchzeit und Metadaten
hardware_conditions.py; summarize_periodic_results.py	Logische Ressourcen und historische Zusammenfassungen; keine physische Zeitmessung
legacy/: model.py; sorting.py; evaluate.py; validate.py	Historisches gröberes Ausgangsmodell und zugehörige Validierung
qant_sorting/report.py; report_assets/thesis_report.md	Aktueller Gesamtbericht aus Text, Quellen und gespeicherten Tabellen
legacy/build_reports.py; report_assets/report_template.html	Historischer Generator; Ausgabe nach tmp/legacy_reports
tests/: Modell-, Metrik-, SDK- und Strukturtests	Gezielte Modell-, Qualitäts-, Backend- und Benchmark-Vertragsprüfungen (alle unter tests/)

13.2 Datenverträge und Fehlerbehandlung

Listen bestehen aus Schlüsseln und eindeutigen Originalindizes. Die Eingabeprüfung kontrolliert Wertebereiche und Formen; Bitonic benötigt Zweierpotenzen. Bei gemessenem Gleichstand entscheidet der Originalindex. Die Ausgabeprüfung trennt gültige Datensatzpermutation, Schlüsselordnung und Stabilität. Widersprüchliche Rangentscheidungen bleiben als ungültige Ausgabe mit −1-Markern sichtbar; es gibt keine digitale Reparatur.

Metadaten enthalten Seeds, Konfigurationen und Quell-/Eingabehashes. Unvollständige historische Archive erfordern eine ausdrückliche Teilauswertung mit ausgewiesenen Lücken. Ein fehlgeschlagener Hardwarepfad wird nicht still durch eine CPU-Ausführung als Hardwareerfolg ersetzt.

13.3 Aktueller Code und historische Dateien

paths.py definiert Repository-Stamm und Eingabeidentität. io.py bündelt CSV-Ausgabe, gestreamte SHA-256-Berechnung und Quellinventar. Gemeinsame Versuchsparameter liegen in experiments/settings.py. Die Komparatoren bleiben wegen ihrer unterschiedlichen wissenschaftlichen Annahmen getrennt. Importe aktueller Module starten keine Versuche oder Dateiausgaben.

legacy enthält die unverändert verschobenen Prototypen und den früheren Berichtsgenerator. requirements enthält die Umgebungen; die identischen Abhängigkeiten von Qualitätsanalyse und gemeinsamem Rauschmodell stehen in analysis.txt. docs/source_layout.json dokumentiert die Pfadzuordnung. Historische Metadaten behalten ihre ursprünglichen Quellhashes.

python -m qant_sorting report erzeugt aus Text, Literatur und gespeicherten Ergebnissen die einzige aktuelle PDF in reports. Frühere Berichte bleiben in der Git-Historie erhalten. Der historische Generator schreibt ausschließlich nach tmp/legacy_reports.

14. Reproduzierbarkeit und Prüfung

14.1 Identität der Eingaben und des SDK

Der verwendete Eingabekorpus data/inputs.npz besitzt den SHA-256-Wert:

79efbd1ee270242cc122d8f9e1848fe07a85077bcf310324b27dd60f8faf11da

Der SDK-Installationshelfer bindet Version 2.3.1 an Commit:

72a2d99f10240b6df3c6d0f636dfa0e2b5d38902

Die bisherigen Versuchsserien beruhen auf der Forschungs-Codebasis fbb418d0b80c04b4fa7c38fe92f56fc7b7317bcd. Die affine Machbarkeitskontrolle vom 9. Oktober 2026 ergänzt den Hauptzweigstand 61b6d74ff7bc9ad10749e073f5f3a329ca8460e7, ohne frühere wissenschaftliche Algorithmen, Eingaben oder Ergebnisse zu verändern. report_provenance.json bewahrt die bisherige Quellen- und Evidenzinventur; affine_evidence.json ergänzt die affinen Ergebnisdateien mit Hashes. Die nachfolgende Paketorganisation basiert auf Commit a750d88b217b0dd6ae732a4e1e5dcb27ade2e75f. Ursprüngliche Quellhashes bleiben als historischer Snapshot erhalten; die PDF-Erzeugung prüft weiterhin die tatsächlich verwendeten Ergebnisdateien. Der zugehörige Git-Commit hält die genaue Fassung fest.

14.2 Getrennte Umgebungen statt stiller Versionsmischung

Die ursprüngliche Referenz, die neueren Qualitäts-/Rauschwerkzeuge, die CPU-Zeitmessung und der Bericht besitzen unterschiedliche Abhängigkeitsdateien. Für einen reproduzierbaren Lauf ist jeweils eine eigene Umgebung sinnvoll. Die historische requirements/reference.txt enthält NumPy 2.5.3; die neueren CPU- und Kontrollauswertungen dokumentieren NumPy 2.3.5. Das ist offenzulegen und nicht durch eine rückwirkende Versionsänderung zu kaschieren.

Die affine CPU-Kontrolle verwendet Python 3.12.14, NumPy 2.3.5, ml-dtypes 0.6.0, cffi 2.1.1, pycparser 3.0 und das hashgeprüfte offizielle CPU-Wheel 2.3.1. requirements/sdk.txt und docs/PERIODIC_FEASIBILITY.md beschreiben die getrennte Einrichtung.

Die CPU-Baseline benötigt requirements/cpu.txt, die gemeinsame Rauschkontrolle requirements/analysis.txt und die Rankingauswertung requirements/analysis.txt. Für die PDF-Erzeugung dient requirements/report.txt. Das SDK wird über den vorgesehenen Installationshelfer in einer kompatiblen Umgebung eingebunden. Laufzeitwerte gelten für die jeweils protokollierte Umgebung; Abweichungen auf anderen Rechnern sind zu erwarten.

14.3 Reproduktionsbefehle

Die Befehle werden im Repository-Stamm ausgeführt. Neue Ausgaben erhalten eigene Verzeichnisse, damit gespeicherte Referenzen nicht überschrieben werden. Zunächst wird die passende Umgebung eingerichtet; danach können die gewünschten Teilschritte getrennt ausgeführt werden.

```
python -m qant_sorting noise --output-dir results/reproduced
python -m qant_sorting verify results/reproduced
python -m qant_sorting install-sdk
python -m qant_sorting sdk
python -m qant_sorting nonlinearity
python -m qant_sorting periodic-pairs --output-file results/periodic_reproduced/pair_checks.json
```

Die SDK-Schritte benötigen den installierten offiziellen CPU-Pfad. Sie liefern keine Hardwaremessung. Neue periodische Ausgaben können mit run_periodic_evaluation.py erzeugt werden; Szenario und

Ausgabeordner müssen zum gewünschten Versuch passen. Die anschließend ausgeführte Qualitätsanalyse benötigt die entsprechenden Originalindex-Ausgaben.

```
python -m qant_sorting affine --output-dir results/affine_periodic_reproduced
python -m qant_sorting common-noise --output-dir results/common_noise_reproduced
python -m qant_sorting cpu --output-dir results/cpu_baseline_reproduced
python -m qant_sorting quality --allow-incomplete --output-dir results/ranking_quality_reproduced
python -m unittest discover -s tests -v
python -m qant_sorting report
```

Der Qualitätsbefehl reproduziert die historischen periodischen Kennzahlen nur, wenn auch die dazugehörigen älteren Roharchive verfügbar sind. --allow-incomplete erlaubt dokumentierte Teilauswertung, beschafft aber keine fehlenden Dateien und füllt keine Lücken. Ein frischer Git-Klon enthält bereits die abgeleiteten historischen Qualitätstabellen. Die PDF-Erzeugung verwendet diese gespeicherten Tabellen und führt keine Sortier- oder Zeitmessung neu aus.

14.4 Was die Prüfungen absichern

Die mathematischen Tests betreffen Signentscheidung, Bindungsregeln, Bitonic-Stufen, Rangbelegung, Quantisierung und die kontrollierten Rauschfälle. Die Rankingmetriken wurden zusätzlich anhand gezielter kleiner Beispiele und 40 SciPy-Vergleiche geprüft. Ergebnisvalidierungen gleichen Eingabehashes, Konfigurationsabdeckung, Zählungen, Kontrollen und End-to-End-Ausgaben ab.

Diese Prüfungen stützen die Implementierung unter ihrem jeweiligen Modell. Sie ersetzen weder einen physikalischen Kennliniennachweis noch die externe Reproduktion der CPU-Zeitmessungen. Die Berichtsprüfung kontrolliert zusätzlich Quellenzuordnung, die unveränderten wissenschaftlichen Daten, Tabellenwerte, lesbare PDF-Seiten und das Vorhandensein genau einer aktuellen Datei im Ordner reports. Für die Paketorganisation wurden die vollständige Referenz mit 288.000 Sortierungen, der lineare CPU-Kontrolllauf mit 48.000 Sortierungen, die gemeinsame Rauschkontrolle und die affine Kontrolle separat reproduziert. Das Prüfprotokoll validation/structure_cleanup_20261009.json dokumentiert den Ergebnisabgleich und die Strukturtests.

15. Hardwarefragen und noch ausstehende Arbeit

15.1 Was öffentlich belegt und was unbekannt ist

Öffentlich belegt sind der untersuchte SDK-Vertrag, die unterschiedliche CPU-/Hardware-Anbindung und Herstellerangaben zum Produkt. Aus diesen Angaben sind aber weder die tatsächlich zulässigen Phasen- und Amplitudenbereiche noch die Komparatorauflösung, Drift, Kanalabhängigkeit, reale tcos-Kennlinie oder Sortierzeit ableitbar. Hinweise auf Systeme an Rechenzentren sind kein Nachweis eines für diese Arbeit verfügbaren externen Zugangs. [36, 37]

Eine Anfragevorlage und ein Messplan liegen vor. Der Entwurf fragt zusätzlich nach einer residenten affinen MVM-/Nichtlinearitätsverbindung und danach, ob die BF16-Produktrundung des CPU-Backends der tatsächlichen Gerätepräzision entspricht. Ein tatsächlich erfolgter Versand, eine Herstellerantwort oder ein nutzbarer NPU-Zugang sind im ausgewerteten Stand nicht dokumentiert. Daher werden keine Herstellerzusagen und keine physikalischen Messungen behauptet.

Tabelle 23. Benötigte Herstellerinformationen und Messungen

Frage	Konkrete Erhebung	Nutzen für das Mapping
Zulässige u-/v-Bereiche	Versionierter Treibervertrag, Einheiten und Kanalnormierung	Legale Phasen und Amplituden
Kennlinie und Nullpunkt	Dichte Sweeps plus verwendete Differenzcodes	u_0, monotones Intervall und Mindestabstand
Auflösung und Rundung	Codehistogramme, DAC-/ADC-Einstellungen	Quantisierung getrennt von BF16
Rauschen und Korrelation	Wiederholungen je Pegel/Kanal; Rohverteilungen und Tails	Physikalische Parameter statt η-Annahme
Drift und Kalibrierung	Zeitgestempelte Referenzen, Last-/Temperaturfolgen	Gemeinsame Fehler und Kalibrierintervall
Kapazität und Kaskade	Bestätigte Dimensionen, Pufferspeicherung und Datenpfad	Aufrufzahl und vermiedene Hostschritte
Zeit und Energie	Cold-/Warm-Läufe, Transfers, vollständige Leistungsspuren	Gleiche Aufgabe gegen CPU/GPU

Grundlage: docs/QANT_MEASUREMENT_PLAN.md. Anfragevorlage vorhanden; kein dokumentierter Messzugang.

15.2 Vorbereiteter Kennliniensweep

measure_periodic_curve.py kann Phasenpunkte, tatsächlich codierte BF16-Werte, Wiederholungen, Mittelwerte, Streuungen, Zeitstempel und API-Zeiten protokollieren. Eine historische CPU-Kontrolle verwendete 257 angeforderte Punkte zwischen 0 und π bei Amplitude eins und 20 Wiederholungen. Aufgrund der BF16-Codierung verbleiben 237 verschiedene Phasencodes. Die deterministischen CPU-Ergebnisse charakterisieren keine physikalische Drift oder Auflösung; das zugehörige alte Rohverzeichnis ist im aktuellen Git-Stand nicht enthalten.

Für eine reale Messung sind zuerst Herstellerfreigaben zu Wertebereich, Dimensionen und Treiber einzuholen. Danach sind insbesondere die tatsächlich verwendeten Schlüsseldifferenzen, Punkte nahe dem Gleichstand, Vorwärts-/Rückwärtssweeps und zeitlich eingestreute Referenzen zu messen. Rohwerte und Fehlerstatus werden erhalten; ein einzelner Gaußfit reicht nicht, wenn Ausreißer, Kanalunterschiede oder Korrelationen die Entscheidung bestimmen.

15.3 Faire digitale und hybride Vergleiche

Die CPU-Baseline ist als erster Leistungsanker vorhanden. Ein GPU-Vergleich muss denselben stabilen Ausgabeumfang herstellen und getrennt die geräteinterne Sortierzeit sowie die vollständige Zeit einschließlich Host-Geräte-Transfers messen. Synchronisation, Warm-up, Datentyp, Batchgröße und verwendete Bibliothek sind zu protokollieren. Eine nichtstabile oder ausschließlich Top-k liefernde Implementierung wäre eine andere Aufgabe und müsste entsprechend gekennzeichnet werden.

Für einen NPU-Vergleich werden erst kalibrierte Genauigkeit und anschließend vollständige Laufzeit und Energie benötigt. Direkte elektronische Signentscheidung, linearer SDK-Pfad und eine bestätigte kaskadierte Nichtlinearität bilden getrennte Vergleichsfälle. Nur so lässt sich erkennen, ob ein photonischer Schritt Arbeit einspart oder lediglich eine bereits verfügbare Entscheidung erneut codiert.

15.4 Arbeit auch ohne Hardwarezugang tragfähig halten

Der nächste Schwerpunkt ist die quantitativ belegte Parametrisierung des gemeinsamen Modells oder, falls diese ausbleibt, eine klar begrenzte Sensitivitätsanalyse über dokumentierte Annahmebereiche. Danach folgen eine ergänzende GPU-Baseline bei verfügbarer Plattform und eine Ressourcenanalyse mit expliziten Bedingungen für einen möglichen Vorteil. Historische periodische Roharchive sollten für eine vollständige Archivierung wiederbeschafft oder als dauerhaft fehlend gekennzeichnet werden.

Ohne Hardware bleibt eine belastbare Arbeit möglich: Sie kann mathematische Eignung, softwareseitige Funktionsfähigkeit, Fehlertypen, Qualitätsanforderungen, digitalen Aufwand und bedingte Systemgrenzen nachweisen. Ihre Schlussfolgerung darf dann keine gemessene photonische Beschleunigung, Energieüberlegenheit oder physikalische Rauschrobustheit behaupten.

16. Gesamtbewertung des aktuellen Stands

Die Recherche verbindet klassische optische Sortiernetze mit heutigen hybriden photonischen Prozessoren und einer systematischen Betrachtung von Präzision, Datenbewegung und Energie. Die fünf Architekturen sind unter einem einheitlichen real-RAM-artigen Arbeitsmaß eingeordnet. Unterschiedliche Installationsbreite, Parallelität und Regeneration werden sichtbar, ohne sie mit der Zahl skalarer Rechenoperationen gleichzusetzen.

Für zwei ausgewählte Mappings liegen ausführbarer Code, gespeicherte Eingaben, mathematische Kontrollen und vollständige Sortierauswertungen vor. Der periodische Baustein mit Host-Phasenbildung ist im CPU-Backend funktionsfähig, im vorhandenen Hostablauf jedoch nicht als nützliche Beschleunigung begründet. Die untersuchte affine Alternative verliert bei 8 Bit bereits ohne Rauschen Vergleichsinformation. Sie liefert damit eine konkrete Grenze dieser Umsetzung und eine gezielte Frage an den Hersteller. Die gemeinsame Rauschmodellierung beseitigt einen zentralen Vergleichsfehler der früheren Szenarien: Signalabstand, Rundung und Rauschort müssen gemeinsam interpretiert werden.

Kendall-τ, Abstandsfehler und Recall zeigen, wie stark eine strenge vollständige Korrektheitsquote von einer guten näherungsweisen Rangfolge abweichen kann. Gleichzeitig verhindern Gültigkeits- und Stabilitätsmetriken, dass Rangkonflikte oder vertauschte Duplikate verborgen bleiben. Die CPU-Baseline schafft einen real gemessenen digitalen Bezugspunkt.

Der erreichte Beitrag ist damit eine nachvollziehbare Forschungs- und Evaluierungsgrundlage mit klaren positiven und negativen Ergebnissen. Die verbleibende Frage ist nicht, ob sich Sortierung formal in photonisch geeignete Funktionen zerlegen lässt, sondern ob ein belegter vollständiger Datenpfad diese Funktionen mit ausreichender Qualität und vertretbarem Systemaufwand ausführt.

Anhang A. Literaturverzeichnis und Einordnung

Die folgenden Einträge decken die in den beiden Präsentationen, der bereitgestellten Gesamtdokumentation und den beigefügten Literatursammlungen untersuchten Arbeiten ab. Zu jedem Werk wird seine Rolle für die Masterarbeit benannt. Einzel-PDF, eingebetteter Originaltext, Auswertung in einer Literatursammlung und ergänzende offizielle Dokumentation werden unterschieden. Eine Erwähnung in einer Sammlung bedeutet keine unabhängige Reproduktion der darin beschriebenen Hardware.

[1] Peter L. McMahon (2023). **The physics of optical computing**.

doi.org/10.1038/s42254-023-00645-5

Grundlage: Einzel-PDF 18; bereitgestellte arXiv-Fassung 2308.00088. **Einordnung:** Physikalische Vorteile, Grenzen und Systemeinbettung optischer Rechenoperationen; kein Sortierbenchmark.

[2] Andrés Macho-Ortiz, Daniel Pérez-López, José Azaña, José Capmany (2023; Preprint 2022). **Analog Programmable-Photonic Computation**.

doi.org/10.1002/lpor.202200360

Grundlage: Einzel-PDF 25. **Einordnung:** Anbits, programmierbare lineare Gatter und Rückkopplung als APC-Grundlage; klassische optische Verarbeitung, keine Q.ANT-API-Spezifikation.

[3] C. D. Thompson (1979). **Area-Time Complexity for VLSI**.

doi.org/10.1145/800135.804401

Grundlage: Einzel-PDF 24. **Einordnung:** Physische Flächen-Zeit-Abwägung und Kommunikation; die DFT-Schranke wird nicht als Sortierschranke übernommen.

[4] H. J. Caulfield (1991). **Space-Time Complexity in Optical Computing**.

doi.org/10.1007/BF01937172

Grundlage: Einzel-PDF 19. **Einordnung:** Räumliche, zeitliche und Fan-in-Ressourcen als Grenze einfacher Aussagen über konstante optische Zeit.

[5] Yandong Li, Francesco Monticone (2025). **The spatial complexity of optical computing: toward space-efficient design**.

doi.org/10.1038/s41467-025-63453-8

Grundlage: Einzel-PDF 17. **Einordnung:** Raumbedarf linearer Operatoren und nichtlokaler Kommunikation; nur bedingt auf die nichtlineare Gesamtsortierung übertragbar.

[6] K. E. Batcher (1968). **Sorting Networks and Their Applications**.

doi.org/10.1145/1468075.1468121

Grundlage: Einzel-PDF 20. **Einordnung:** Grundlage der festen Bitonic-Stufen und Vergleichszahlen; kein physikalisches Zeitmodell.

[7] Charles W. Stirk, Ravindra A. Athale (1988). **Sorting with optical compare-and-exchange modules**.

doi.org/10.1364/AO.27.001721

Grundlage: Einzel-PDF 07; auch Sortiersammlung 05. **Einordnung:** Räumliche C&E-Pipeline, Implementierungsvarianten und Regeneration; Ausgangspunkt der aktiven Bitonic-Adaption.

[8] F. R. Beyette, Jr., P. A. Mitkas, S. A. Feld, C. W. Wilmsen (1994). **Bitonic sorting using an optoelectronic recirculating architecture**.

doi.org/10.1364/AO.33.008164

Grundlage: Einzel-PDF 02. **Einordnung:** Wiederverwendung von Smart-Pixel-Arrays; trennt vollständige Antwortzeit und Ergebnisintervall.

[9] M. P. Y. Desmulliez et al. (1995). **Perfect-shuffle interconnected bitonic sorter: optoelectronic design**.

doi.org/10.1364/AO.34.005077

Grundlage: Einzel-PDF 06. **Einordnung:** Perfect Shuffle, Kontrollmasken und bitserielle Zyklen; kein zusätzlicher moderner SDK-Hardwareversuch.

[10] Ahmed Louri, James A. Hatch, Jr., Jongwhoa Na (1995). **Constant-time parallel sorting algorithm and its optical implementation using smart pixels**.

doi.org/10.1364/AO.34.003087

Grundlage: Einzel-PDF 03. **Einordnung:** Broadcast-and-Compare, Rangbildung und Gleichstände; konstante breite Ebenenzahl mit quadratischen Ressourcen.

[11] Kenji Mizutani et al. (2025; online 2024). **In-network stable radix sorter using many FPGAs with high-bandwidth photonics [Invited]**.

doi.org/10.1364/JOCN.530695

Grundlage: Einzel-PDF 04. **Einordnung:** FPGA/HBM-Sortierung mit photonischen Links; Acht-FPGA-Messung und getrennte Skalierungsprojektion.

[12] Sufi R. Ahmed et al. (2025). **Universal photonic artificial intelligence acceleration**.

doi.org/10.1038/s41586-025-08854-x

Grundlage: Einzel-PDF 16. **Einordnung:** Hybrider integrierter Beschleuniger als Referenz für digitale Peripherie und Wandlungskosten; kein Q.ANT-System.

[13] Xiansong Meng et al. (2025). **Digital-analog hybrid matrix multiplication processor for optical neural networks**.

doi.org/10.1038/s41467-025-62586-0

Grundlage: Einzel-PDF 08. **Einordnung:** Hybride Präzisionssteigerung bei Matrixoperationen; zusätzliche digitale Arbeit muss mitgezählt werden.

[14] Zichao Zhao et al. (2025). **Efficient hardware error correction with hybrid on-offline configuration algorithm for optical processor**.

doi.org/10.1038/s42005-025-02247-2

Grundlage: Einzel-PDF 09. **Einordnung:** Kalibrierung durch Offline-/Online-Konfiguration; motiviert die getrennte Behandlung systematischer Fehler.

[15] Huize Li, Dan Chen, Tulika Mitra (2025). **Hybrid Photonic-digital Accelerator for Attention Mechanism (HyAtten)**.

arxiv.org/abs/2501.11286

Grundlage: Einzel-PDF 10; Preprint. **Einordnung:** Selektive digitale Verarbeitung bei beschränkter Analogpräzision; mögliche Anregung, im Sortercode nicht implementiert.

[16] Yaowen Hu et al. (2025). **Integrated lithium niobate photonic computing circuit based on efficient and high-speed electro-optic conversion**.

doi.org/10.1038/s41467-025-62635-8

Grundlage: Einzel-PDF 11. **Einordnung:** TFLN-basierte Modulation und Rechenoperationen; kein Beleg für frei kaskadierbare Q.ANT-Funktionen.

[17] Hailong Gong et al. (2026). **LightMat-HP: A Photonic–Electronic System for Accelerating General Matrix Multiplication With Configurable Precision**.

arxiv.org/abs/2604.12278

Grundlage: Einzel-PDF 12; arXiv v1, 14. April 2026. **Einordnung:** Niederbitrige photonische Teilprodukte und digitale Rekonstruktion; der DOI-Platzhalter der bereitgestellten Fassung wird nicht als Publikationsnachweis verwendet.

[18] Hajime Sakata (2001). **Photonic analog-to-digital conversion by use of nonlinear Fabry–Perot resonators**.

doi.org/10.1364/AO.40.000240

Grundlage: Einzel-PDF 13. **Einordnung:** Nichtlineare optische Wandlung als alternativer Baustein; kein Parameterlieferant für die verwendete Hardware.

[19] Nathaniel Nauman, James Robinson, Yuyang Wang, Kaylx Jang, Xiang Meng, Keren Bergman (2025). **Photonic Analog-to-Digital Architecture for Accelerating Multiply-Accumulate Operations**.

doi.org/10.1364/OFC.2025.W3D.5

Grundlage: Einzel-PDF 14; Veröffentlichung OFC 2025, W3D.5, zusätzlich beim Verlag geprüft. **Einordnung:** Photonische ADC-Architektur für MAC; Motivation der Wandlerschnittstelle, keine Q.ANT-Messung.

[20] Jing Xu et al. (2025). **Progress in silicon-based reconfigurable and programmable all-optical signal processing chips**.

doi.org/10.1007/s12200-025-00154-6

Grundlage: Einzel-PDF 15. **Einordnung:** Übersicht über programmierbare Signalverarbeitung und Regeneration; Funktionsvielfalt setzt konkrete Bauelemente voraus.

[21] Raúl López-March, Andrés Macho-Ortiz, Francisco Javier Fraile-Peláez, José Capmany (2026). **Noise in analog programmable-photonic computation**.

arxiv.org/abs/2604.24541

Grundlage: Einzel-PDF 22; arXiv v1, 27. April 2026. **Einordnung:** Physikalische Störquellen und Fehlerfortpflanzung; Beispielwerte gehören zur dortigen PIP-Anordnung und sind keine Q.ANT-Kalibrierung.

[22] Niklas Bahr, Daniel Steinmeyer, Wolfram Pernice (2026). **Energy Efficiency in Analog Photonic Processors: Conversions and Losses at Scale**.

doi.org/10.23919/ISC.2026.11520497

Grundlage: Einzel-PDF 23; bildbasierte Fassung, zusätzlich OCR gelesen. **Einordnung:** Systemkostenmodell für Konversion, Gewichte, Speicher und Verluste; digitale Steuerlogik ausgenommen, kein Sortier-Energiebefund.

[23] Hyoseok Park, Yeonsang Park (2026). **PRISM: Photonic Similarity Engine for KV Cache Block Selection in Long-Context LLM Inference**.

arxiv.org/abs/2603.21576

Grundlage: Einzel-PDF 21; v2 vom 25. März 2026 mit Supplement. **Einordnung:** Photonische Scorebildung und elektronische Auswahl; Anwendungsmotivation für Recall, keine vollständige stabile Integer-Sortierung.

[24] Morris Rubinoff (1953). **Analogue vs. Digital Computers—A Comparison**.

ieeexplore.ieee.org/document/4051190

Grundlage: Recherchemappe 01, ab S. 3. **Einordnung:** Historische Unterscheidung analoger und digitaler Rechenprinzipien; keine quantitative moderne Hardwarebaseline.

[25] Guo Qi Zhang, Alfred van Roosmalen (Hrsg.) (2009). **More than Moore: Creating High Value Micro/Nanoelectronics Systems**.

doi.org/10.1007/978-0-387-75593-9

Grundlage: Recherchemappe 01, S. 2; Buchverweis. **Einordnung:** Technologischer Kontext jenseits reiner Transistorskalierung; kein eigenständiger Sortier- oder Photonikversuch.

[26] H. John Caulfield, Shlomi Dolev (2010). **Why future supercomputing requires optics**.

doi.org/10.1038/nphoton.2010.94

Grundlage: Recherchemappe 01, ab S. 14. **Einordnung:** Motivation optischer Kommunikation und Parallelität; die perspektivische Argumentation ist kein Nachweis für den vorliegenden Workload.

[27] Francesco Morichetti (2025). **High-performance analog signal processing with photonic integrated circuits**.

doi.org/10.1038/s41377-025-01806-0

Grundlage: Recherchemappe 01, ab S. 18; Kommentarartikel. **Einordnung:** Einordnung integrierter analoger Signalverarbeitung; Kommentare und darin besprochene Primärdemonstratoren bleiben getrennt.

[28] H. Zhou et al. (2022). **Photonic matrix multiplication lights up photonic accelerator and beyond**.

doi.org/10.1038/s41377-022-00717-8

Grundlage: Recherchemappe 01, ab S. 19; Präsentationsliteratur. **Einordnung:** Übersicht der Matrixmultiplikation über räumliche, interferometrische und Wellenlängenverfahren; Grundlage der MVM-Einordnung.

[29] P. Bruce Berra, Arif Ghafoor, Pericles A. Mitkas, Slawomir J. Marcinkowski, Mohsen Guizani (1989). **The Impact of Optics on Data and Knowledge Base Systems**.

ieeexplore.ieee.org/document/43408

Grundlage: Recherchemappe 01, ab S. 28. **Einordnung:** Früher Kontext optischer Datenbankoperationen und Datenbewegung; keine direkt vergleichbare aktuelle Messung.

[30] Yinyi Liu et al. (2022). **A Reliability Concern on Photonic Neural Networks**.

doi.org/10.23919/DATE54114.2022.9774661

Grundlage: Recherchemappe 01, ab S. 45. **Einordnung:** Architekturspezifische Zuverlässigkeit und Übersprechen; keine universelle Modenzahl- oder Rauschgrenze für Q.ANT.

[31] Bibliografischer Eintrag aus der Recherchemappe (2025). **Integrated PCM-based DAC/ADC Design for Energy-Efficient Photonic Computing Systems**.

doi.org/10.1117/12.3066474

Grundlage: Recherchemappe 01, ab S. 108; Auswertungsnotizen. **Einordnung:** PCM-basierte Konversion als weiterer Ansatz; nicht separat reproduziert. Prozentuale Einsparungen aus den Notizen werden nicht als eigene oder Q.ANT-bezogene Befunde übernommen.

[32] Nikolaos Farmakidis, Bowei Dong, Harish Bhaskaran (2024). **Integrated photonic neuromorphic computing: opportunities and challenges**.

doi.org/10.1038/s44287-024-00050-9

Grundlage: Recherchemappe 01, ab S. 363. **Einordnung:** Systematische Einordnung hybrider neuronaler Photonik und Schnittstellen; Motivation vollständiger Ausführungsgrenzen.

[33] M. A. Al-Qadasi, L. Chrostowski, B. J. Shastri, S. Shekhar (2022). **Scaling Up Silicon Photonic-Based Accelerators: Challenges and Opportunities**.

doi.org/10.1063/5.0070992

Grundlage: Recherchemappe 01, ab S. 377; Preprint 2109.08025. **Einordnung:** Optische und elektrische Linkbudgets, Skalierung und Energiekosten; Parameter müssen zum jeweiligen Aufbau passen.

[34] G. Giamougiannis et al. (2023). **Analog nanophotonic computing going practical: silicon photonic deep learning engines for tiled optical matrix multiplication with dynamic precision**.

doi.org/10.1515/nanoph-2022-0423

Grundlage: Recherchemappe 01, ab S. 393. **Einordnung:** Tiling und dynamische Präzision als Systemmethoden; keine automatische Garantie für exakte Vergleichsentscheidungen.

[35] Qingyi Zhou et al. (2026). **Quantum Nonlinearity for Optical Neural Computing**.

arxiv.org/abs/2601.01690

Grundlage: Recherchemappe 01, ab S. 413; aktueller arXiv-Eintrag zusätzlich geprüft. **Einordnung:** Quantenemitterbasierte Nichtlinearität in einer anderen Architektur; nicht mit klassischem APC oder einem verfügbaren Q.ANT-Kaskadenpfad gleichzusetzen.

[36] Q.ANT GmbH (SDK 2.3.1). **Q.ANT Native Computing Toolkit: festgelegter Quellstand**.

github.com/Q-ANT-GmbH/qant_native_computing_toolkit/tree/72a2d99f10240b6df3c6d0f636dfa0e2b5d38902

Grundlage: Offizieller SDK-Quellstand; Prüfsummen und CPU-Paket im Projekt. **Einordnung:** API-Verträge, BF16-Formate, Host-ReLU sowie CPU-/Hardware-Unterscheidung; kein offener vollständiger Hardwaretreiber.

[37] Q.ANT GmbH (abgerufen 7. Oktober 2026). **Photonic Computing: Produktinformationen**.

qant.com/photonic-computing/

Grundlage: Offizielle Produktseite; ergänzend Generation-2-Broschüre vom Juni 2026. **Einordnung:** Herstellerangaben zu Schnittstelle, Leistung und Produkt; keine eigenen Komparator-, Kennlinien- oder Sortiermessungen.

[38] NumPy Developers (Dokumentation 2.3). **numpy.argsort und numpy.take_along_axis**.

numpy.org/doc/2.3/reference/generated/numpy.argsort.html

Grundlage: Offizielle API-Dokumentation; installierte Benchmarkversion 2.3.5. **Einordnung:** Stabile indirekte Sortierung und Einsammeln der Originalwerte; die konkreten Zeitwerte stammen ausschließlich aus den gespeicherten eigenen Messungen.

[39] SciPy Developers (Dokumentation). **scipy.stats.kendalltau**.

docs.scipy.org/doc/scipy/reference/generated/scipy.stats.kendalltau.html

Grundlage: Offizielle Statistikdokumentation; unabhängige Implementierungskontrolle. **Einordnung:** Definition und Bindungskorrektur von Kendall-τ-b; keine Ersetzung der getrennten Gültigkeits- und Stabilitätsmetriken.

[40] Nikita Stroev, Natalia G. Berloff (2023). **Analog Photonics Computing for Information Processing, Inference and Optimisation**.

arxiv.org/abs/2301.11760

Grundlage: Präsentation P2, S. 25; arXiv-Eintrag ergänzend geprüft. **Einordnung:** Übersicht analoger photonischer Plattformen, Optimierung und Inferenz. Die Präsentation darf daraus keine universell fehlerfreie physikalische Sortierung ableiten.

Anhang B. Quellenbestand und Ergebnisprovenienz

B.1 Bereitgestellte Dokumente

Für die Zusammenführung wurden beide Präsentations-PDFs, der bisherige ausführliche Bericht und die 25 bereitgestellten Recherche-PDFs ausgewertet. Die zwei umfangreichen Literatur-/Sortiersammlungen enthalten zusätzliche Originaltexte beziehungsweise Notizen und überschneiden sich mit den Einzel-PDFs. Die folgende Liste dokumentiert Dateien, nicht 25 voneinander unabhängige Forschungsergebnisse. Die ursprünglichen Volltexte werden nicht in das öffentliche Repository kopiert.

Tabelle 24. Ausgewertete bereitgestellte Dateien

ID	Dokument	Seiten
01	Recherchemappe: Analog Photonic Computing, hybride Architekturen und Sortierung	460
02	Bitonic sorting using an optoelectronic recirculating architecture	9
03	Constant-time parallel sorting algorithm and its optical implementation using smart pixels	10
04	In-network stable radix sorter using many FPGAs with high-bandwidth photonics [Invited]	12
05	Sammlung optischer Sortierverfahren	51
06	Perfect-shuffle interconnected bitonic sorter: optoelectronic design	14
07	Sorting with optical compare-and-exchange modules	6
08	Digital-analog hybrid matrix multiplication processor for optical neural networks	11
09	Efficient hardware error correction with hybrid on-offline configuration algorithm for optical processor	9
10	Hybrid Photonic-digital Accelerator for Attention Mechanism (HyAtten)	7
11	Integrated lithium niobate photonic computing circuit based on efficient and high-speed electro-optic conversion	11
12	LightMat-HP: A Photonic–Electronic System for Accelerating General Matrix Multiplication With Configurable Precision	28
13	Photonic analog-to-digital conversion by use of nonlinear Fabry–Perot resonators	9
14	Photonic Analog-to-Digital Architecture for Accelerating Multiply-Accumulate Operations	3
15	Progress in silicon-based reconfigurable and programmable all-optical signal processing chips	53
16	Universal photonic artificial intelligence acceleration	8
17	The spatial complexity of optical computing: toward space-efficient design	11
18	The physics of optical computing	31
19	Space-Time Complexity in Optical Computing	6
20	Sorting Networks and Their Applications	8

ID	Dokument	Seiten
21	PRISM: Photonic Similarity Engine for KV Cache Block Selection in Long-Context LLM Inference	27
22	Noise in analog programmable-photonic computation	27
23	Energy Efficiency in Analog Photonic Processors: Conversions and Losses at Scale	9
24	Area-Time Complexity for VLSI	8
25	Analog Programmable-Photonic Computation	15
P1	Präsentation: Hybride optoelektronische Sortierarchitekturen	44
P2	Präsentation: Machbarkeitsstudie APC für Sortierung	29
B1	Bisherige ausführliche Forschungsdokumentation mit korrigiertem Komplexitätsmodell	50

Kennungen 01–25 entsprechen den bereitgestellten Recherchedateien; P1/P2 den Präsentationen, B1 dem ausführlichen Bericht. Dateinamen und vollständige Hashes stehen in report_provenance.json.

B.2 Verwendete Ergebnisbestände

Tabelle 25. Ergebnisdateien und Verfügbarkeit

ID / Verzeichnis	Inhalt	Verfügbarkeit im Git-Stand
D1: results/reference	Sechs η; Accuracy, Ausgaben, Fehler und Metadaten	Vollständige Referenzdateien
D2: results/sdk_reference	Rauschfreie lineare SDK-Ausgaben	Ergebnisse und Metadaten
D3: results/periodic/pair_checks.json	Alle 4-/8-Bit-Paare nach Referenzkorrektur	Paarprüfung vorhanden; ältere Sortier-Roharchive fehlen
D4: results/ranking_quality_20261007	summary, recall, distance, coverage, Prüfungen	Abgeleitete Daten; fünf historische Ausgabelücken
D5: results/common_noise_20261007	Paarfälle, Skalen, Sortierausgaben und Qualität	Vollständige gespeicherte Kontrollserie
D6: results/cpu_baseline_20261007	Einzelzeiten, Aggregation, Umgebung und Validierung	Gemessene CPU-Baseline
data/inputs.npz	24 Datensätze mit je 1.000 Eingaben	Gemeinsamer Korpus mit festem Datei-Hash
D7: results/affine_periodic_20261009	Alle Paarscores, 3.600 Sortierausgaben, Zählungen und SDK-Quellprüfung	Neue CPU-Kontrolle; keine Hardwaremessung
D8: results/louri_reference_20261024	Quellengleichungen, rauschfreie Übereinstimmung, 36.000 Sensitivitätsläufe	Ausgaben, Ränge, Qualitätsmaße, Annahmen und Hashes gespeichert
validation; docs	Prüfprotokolle, Methoden und Hardwaregrenzen	Versionsabhängige Belege; Zeitstempel beachten

Die maschinenlesbare Datei report_assets/report_provenance.json enthält SHA-256-Prüfsummen der bereitgestellten Quellen, des Berichtstextes und der für diesen Bericht verwendeten Ergebnisdateien. Die Hashes erlauben eine Zuordnung zu einer konkreten Fassung; sie ersetzen keine öffentlich

verfügbare Kopie einer fehlenden Quelle. Der bisherige MANIFEST.json im Repository gehört zum historischen Projektstand und ist kein aktuelles Gesamtmanifest dieser Konsolidierung.

Anhang C. Begriffe und Leseschlüssel

Tabelle 26. Zentrale Begriffe

Begriff	Bedeutung in diesem Bericht
APC / Anbit	Analog Programmable-Photonic Computation / zweikomponentige klassische optische Informationseinheit
MVM / GEMM	Matrix-Vektor-Multiplikation / allgemeine Matrix-Matrix-Multiplikation
C&E	Compare-and-Exchange: vergleichen und Originaldatensätze gegebenenfalls tauschen
T / physische Zeit	Skalare real-RAM-artige Gesamtarbeit / separat gemessene oder modellierte Sekunden
S / H / D	Lebende logische Wörter / installierte Verarbeitungspositionen / unregenerierte optische Tiefe
Ω / Θ / O	Untere / passende untere und obere / obere asymptotische Schranke
Δ / η / σ	Schlüssel- oder Signalraster / dimensionsloser Rauschfaktor / Standardabweichung in Signaleinheiten
ADC / DAC / ENOB	Analog-Digital-Wandler / Digital-Analog-Wandler / effektive Bitzahl einer definierten Messung
BF16	Bfloat16-Speicher-/Schnittstellenformat; keine Zusage effektiver Analogpräzision
tcos / CPU-Kosinus	Gerätespezifische periodische Funktion / separat untersuchter CPU-Ersatz
Nullbereich / Referenz	Toleriertes Gleichstandsintervall / abgezogener Ausgangswert für d = 0
Gültigkeit / Stabilität	Jeden Originaldatensatz genau einmal ausgeben / Originalreihenfolge gleicher Schlüssel bewahren
Kendall-τ-b / Recall@k	Bindungskorrigierte Rangübereinstimmung / Anteil korrekt ausgewählter Top-k-Elemente
RIN / O/E/O	Relative Intensitätsrauschdichte / optisch-elektronisch-optischer Übergang

Weitere methodische Einzelheiten stehen in docs/METHOD.md, docs/RANKING_QUALITY.md, docs/COMMON_NOISE_MODEL.md, docs/CPU_BASELINE.md und docs/QANT_MEASUREMENT_PLAN.md. docs/CODE_GUIDE.md erschließt die Module. Der vorliegende Bericht verbindet diese Details zu einer durchgehenden Darstellung; die ausführbaren Dateien und gespeicherten Daten bleiben die maßgebliche Grundlage für die Reproduktion.